

ГЛАВА 3

ТИПЫ `STRING`, `VECTOR` И МАССИВЫ

В этой главе...

3.1. Пространства имен и объявления <code>using</code>	124
3.2. Библиотечный тип <code>string</code>	126
3.3. Библиотечный тип <code>vector</code>	141
3.4. Знакомство с итераторами	153
3.5. Массивы	161
3.6. Многомерные массивы	177
Резюме	183
Термины	183

Кроме встроенных типов, рассмотренных в главе 2, язык C++ предоставляет богатую библиотеку абстрактных типов данных. Важнейшими библиотечными типами являются тип `string`, поддерживающий символьные строки переменной длины, и тип `vector`, определяющий коллекции переменного размера. С типами `string` и `vector` связаны типы, известные как *итераторы* (`iterator`). Они используются для доступа к символам строк и элементам векторов.

Типы `string` и `vector`, определенные в библиотеке, являются абстракциями более простого встроенного типа массива. Эта главы посвящена массивам и введению в библиотечные типы `vector` и `string`.

Встроенные типы, рассмотренные в главе 2, определены непосредственно языком C++. Эти типы представляют средства, которые сами по себе присущи большинству компьютеров, такие как числа или символы. Стандартная библиотека определяет множество дополнительных типов, высокоуровневый характер которых аппаратными средствами компьютеров, как правило, не реализуется непосредственно.

В данной главе представлены два важнейших библиотечных типа: `string` и `vector`. Тип `string` — это последовательность символов переменной длины. Тип `vector` содержит последовательность объектов указанного типа переменной длины. Мы также рассмотрим встроенный тип массива. Как и другие встроенные типы, массивы представляют возможности аппаратных

средств. В результате массивы менее удобны в использовании, чем библиотечные типы string и vector.

Однако, прежде чем начать исследование библиотечных типов, рассмотрим механизм, упрощающий доступ к именам, определенным в библиотеке.



3.1. Пространства имен и объявления using

До сих пор имена из стандартной библиотеки упоминались в программах явно, т.е. перед каждым из них было указано имя пространства имен std. Например, при чтении со стандартного устройства ввода применялась форма записи `std::cin`. Здесь использован *оператор области видимости* `::` (см. раздел 1.2, стр. 33). Он означает, что имя, указанное в правом операнде оператора, следует искать в области видимости, указанной в левом операнде. Таким образом, код `std::cin` означает, что используемое имя `cin` определено в пространстве имен `std`.

При частом использовании библиотечных имен такая форма записи может оказаться чересчур громоздкой. К счастью, существуют и более простые способы применения членов пространств имен. Самый надежный из них — *объявление using* (*using declaration*). Другие способы, позволяющие упростить использование имен из других пространств, рассматриваются в разделе 18.2.2 (стр. 991).

Объявление `using` позволяет использовать имена из другого пространства имен без указания префикса `имя_пространства_имен::`. Объявление `using` имеет следующий формат:

```
using пространствоимен::имя;
```

После того как объявление `using` было сделано один раз, к указанному в нем имени можно обращаться без указания пространства имен.

```
#include <iostream>
// объявление using; при использовании имени cin теперь подразумевается,
// что оно принадлежит пространству имен std
using std::cin;
int main()
{
    int i;
    cin >> i;           // ok: теперь cin - синоним std::cin
    cout << i;           // ошибка: объявления using нет; здесь нужно указать
                        // полное имя
    std::cout << i;      // ok: явно указано применение cout из
                        // пространства имен std
    return 0;
}
```

Для каждого имени необходимо индивидуальное объявление using

Каждое объявление using применяется только к одному элементу пространства имен. Это позволяет жестко задавать имена, используемые в каждой программе. Например, программу из раздела 1.2 (стр. 31) можно переписать следующим образом:

```
#include <iostream>
// объявления using для имен из стандартной библиотеки
using std::cin;
using std::cout; using std::endl;
int main()
{
    cout << "Enter two numbers:" << endl;
    int v1, v2;
    cin >> v1 >> v2;
    cout << "The sum of " << v1 << " and " << v2
        << " is " << v1 + v2 << endl;
    return 0;
}
```

Объявления using для имен cin, cout и endl означают, что их можно теперь использовать без префикса std::. Напомню, что программы C++ позволяют поместить каждое объявление using в отдельную строку или объединить в одной строке несколько объявлений. Важно не забывать, что для каждого используемого имени необходимо отдельное объявление using, и каждое из них должно завершаться точкой с запятой.

Заголовки не должны содержать объявлений using

Код в заголовках (см. раздел 2.6.3, стр. 116) обычно не должен использовать объявления using. Дело в том, что содержимое заголовка копируется в текст программы, в которую он включен. Если в заголовке есть объявление using, то каждая включающая его программа получает то же объявление using. В результате программа, которая не намеревалась использовать определенное библиотечное имя, может случайно столкнуться с неожиданным конфликтом имен.

Примечание для читателя

Начиная с этого момента подразумевается, что во все примеры включены объявления using для имен из стандартной библиотеки. Таким образом, в тексте и примерах кода далее упоминается cin, а не std:: cin.

Кроме того, для экономии места в примерах кода не будем показывать далее объявления using и необходимые директивы #include. В табл. А.1 (стр. 1077) приложения А приведены имена и соответствующие заголовки стандартной библиотеки, которые использованы в этой книге.



Читатели не должны забывать добавить соответствующие объявления `#include` и `using` в свои примеры перед их компиляцией.

Упражнения раздела 3.1

Упражнение 3.1. Перепишите упражнения из разделов 1.4.1 (стр. 39) и 2.6.2 (стр. 116), используя соответствующие объявления `using`.



3.2. Библиотечный тип `string`

Строка (`string`) — это последовательность символов переменной длины. Чтобы использовать тип `string`, необходимо включить в код заголовок `string`. Поскольку тип `string` принадлежит библиотеке, он определен в пространстве имен `std`. Наши примеры подразумевают наличие следующего кода:

```
#include <string>
using std::string;
```

В этом разделе описаны наиболее распространенные операции со строками; а дополнительные операции рассматриваются в разделе 9.5 (стр. 462).



Кроме определения операций, предоставляемых библиотечными типами, стандарт налагает также требования на эффективность их конструкторов. В результате библиотечные типы оказались весьма эффективны в использовании.



3.2.1. Определение и инициализация строк

Каждый класс определяет, как могут быть инициализированы объекты его типа. Класс может определить много разных способов инициализации объектов своего типа. Каждый из способов отличается либо количеством предоставляемых инициализаторов, либо типами этих инициализаторов. Список наиболее распространенных способов инициализации строк приведен в табл. 3.1, а некоторые из примеров приведены ниже.

```
string s1;           // инициализация по умолчанию; s1 - пустая строка
string s2 = s1;      // s2 - копия s1
string s3 = "hiya";  // s3 - копия строкового литерала
string s4(10, 'c');  // s4 - ccccccccc
```

Инициализация строки по умолчанию (см. раздел 2.2.1, стр. 77) создает пустую строку; т.е. объект класса `string` без символов. Когда предоставляется строковый литерал (см. раздел 2.1.3, стр. 70), во вновь созданную строку копируются символы этого литерала, исключая завершающий нулевой символ. При предоставлении количества и символа строка содержит указанное количество экземпляров данного символа.

Таблица 3.1. Способы инициализации объекта класса `string`

<code>string s1</code>	Инициализация по умолчанию; <code>s1</code> — пустая строка
<code>string s2(s1)</code>	<code>s2</code> — копия <code>s1</code>
<code>string s2 = s1</code>	Эквивалент <code>s2(s1)</code> , <code>s2</code> — копия <code>s1</code>
<code>string s3("value")</code>	<code>s3</code> — копия строкового литерала, нулевой символ не включен
<code>string s3 = "value"</code>	Эквивалент <code>s3("value")</code> , <code>s3</code> — копия строкового литерала
<code>string s4(n, 'c')</code>	Инициализация переменной <code>s4</code> символом <code>'c'</code> в количестве <code>n</code> штук

Прямая инициализация и инициализация копией

В разделе 2.2.1 (стр. 77) упоминалось, что язык C++ поддерживает несколько разных форм инициализации. Давайте на примере класса `string` начнем изучать, чем эти формы отличаются друг от друга. Когда переменная инициализируется с использованием знака `=`, компилятор просит скопировать инициализирующий объект в создаваемый объект, т.е. выполнить *инициализацию копией* (copy initialization). В противном случае без знака `=` осуществляется *прямая инициализация* (direct initialization).

Когда имеется одиночный инициализатор, можно использовать и прямую форму, и инициализацию копией. Но при инициализации переменной несколькими значениями, как при инициализации переменной `s4` выше, следует использовать прямую форму инициализации.

```
string s5 = "hiya"; // инициализация копией
string s6("hiya"); // прямая инициализация
string s7(10, 'c'); // прямая инициализация; s7 - ccccccccc
```

Если необходимо использовать несколько значений, можно применить косвенную форму инициализации копией при явном создании временного объекта для копирования.

```
string s8 = string(10, 'c'); // инициализация копией; s8 - ccccccccc
```

Инициализатор строки `s8` — `string(10, 'c')` — создает строку заданного размера, заполненную указанным символьным значением, а затем копирует ее в строку `s8`. Это эквивалентно следующему коду:

```
string temp(10, 'c'); // temp - ccccccccc
string s8 = temp;    // копировать temp в s8
```

Хотя используемый для инициализации строки `s8` код вполне допустим, он менее читабелен и не имеет никаких преимуществ перед способом, которым была инициализирована переменная `s7`.



3.2.2. Операции со строками

Наряду с определением способов создания и инициализации объектов класс определяет также операции, которые можно выполнять с объектами класса. Класс может определить обладающие именем операции, такие как функция `isbn()` класса `Sales_item` (см. раздел 1.5.2 стр. 51). Класс также может определить то, что означают различные символы операторов, такие как `<<` или `+`, когда они применяются к объектам класса. Наиболее распространенные операции класса `string` приведены в табл. 3.2.

Таблица 3.2. Операции класса `string`

<code>os << s</code>	Выводит строку <code>s</code> в поток вывода <code>os</code> . Возвращает поток <code>os</code>
<code>is >> s</code>	Читает разделенную пробелами строку <code>s</code> из потока <code>is</code> . Возвращает поток <code>is</code>
<code>getline(is, s)</code>	Читает строку ввода из потока <code>is</code> в переменную <code>s</code> . Возвращает поток <code>is</code>
<code>s.empty()</code>	Возвращает значение <code>true</code> , если строка <code>s</code> пуста. В противном случае возвращает значение <code>false</code>
<code>s.size()</code>	Возвращает количество символов в строке <code>s</code>
<code>s[n]</code>	Возвращает ссылку на символ в позиции <code>n</code> строки <code>s</code> ; позиции отсчитываются от 0
<code>s1 + s2</code>	Возвращает строку, состоящую из содержимого строк <code>s1</code> и <code>s2</code>
<code>s1 = s2</code>	Заменяет символы строки <code>s1</code> копией содержимого строки <code>s2</code>
<code>s1 == s2</code> <code>s1 != s2</code>	Строки <code>s1</code> и <code>s2</code> равны, если содержат одинаковые символы. Регистр символов учитывается
<code><, <=, >, >=</code>	Сравнение зависит от регистра и полагается на алфавитный порядок символов

Чтение и запись строк

Как уже упоминалось в главе 1, для чтения и записи значений встроенных типов, таких как `int`, `double` и т.д., используется библиотека `iostream`. Для чтения и записи строк используются те же операторы ввода и вывода.

```
// Обратите внимание: перед компиляцией этот код следует дополнить
// директивами #include и объявлениями using
int main()
{
    string s;           // пустая строка
    cin >> s;           // чтение разделяемой пробелами строки в s
    cout << s << endl; // запись s в поток вывода
    return 0;
}
```

Программа начинается с определения пустой строки по имени `s`. Следующая строка читает данные со стандартного устройства ввода и сохраняет их в переменной `s`. Оператор ввода строк читает и отбрасывает все предваряющие непечатаемые символы (например, пробелы, символы новой строки и табуляции). Затем он читает значащие символы, пока не встретится следующий непечатаемый символ.

Таким образом, если ввести " **Hello World!** " (обратите внимание на предваряющие и завершающие пробелы), фактически будет получено значение "Hello" без пробелов.

Подобно операторам ввода и вывода встроенных типов, операторы строк возвращают как результат свой левый операнд. Таким образом, операторы чтения или записи можно объединять в цепочки.

```
string s1, s2;  
    cin >> s1 >> s2; // сначала прочитать в переменную s1,  
                      // а затем в переменную s2  
    cout << s1 << s2 << endl; // отобразить обе строки
```

Если в этой версии программы осуществить предыдущий ввод, " **Hello World!** ", выводом будет "HelloWorld! ".

Чтение неопределенного количества строк

В разделе 1.4.3 (стр. 41) уже рассматривалась программа, читающая неопределенное количество значений типа `int`. Напишем подобную программу, но читающую строки.

```
int main()  
{  
    string word;  
    while (cin >> word) // читать до конца файла  
        cout << word << endl; // отобразить каждое слово с новой строки  
    return 0;  
}
```

Здесь чтение осуществляется в переменную типа `string`, а не `int`. Условие оператора `while`, напротив, выполняется так же, как в предыдущей программе. Условие проверяет поток после завершения чтения. Если поток допустим, т.е. не встретился символ конца файла или недопустимое значение, выполняется тело цикла `while`. Оно выводит прочитанное значение на стандартное устройство вывода. Как только встречается конец файла (или недопустимый ввод), цикл `while` завершается.

Применение функции `getline()` для чтения целой строки

Иногда игнорировать пробелы во вводе не нужно. В таких случаях вместо оператора `>>` следует использовать функцию `getline()`. Функция `getline()` получает поток ввода и строку. Функция читает предоставленный поток до

первого символа новой строки и сохраняет прочитанное, *исключая* символ новой строки, в своем аргументе типа string. Встретив символ новой строки, даже если это первый символ во вводе, функция `getline()` прекращает чтение и завершает работу. Если символ новой строки во вводе первый, то возвращается пустая строка.

Подобно оператору ввода, функция `getline()` возвращает свой аргумент типа `istream`. В результате функцию `getline()` можно использовать в условии, как и оператор ввода (см. раздел 1.4.3, стр. 41). Например, предыдущую программу, которая выводила по одному слову в строку, можно переписать так, чтобы она вместо этого выводила всю строку:

```
int main()
{
    string line;
    // читать строки до конца файла
    while (getline(cin, line))
        cout << line << endl;
    return 0;
}
```

Поскольку переменная `line` не будет содержать символа новой строки, его придется вывести отдельно. Для этого, как обычно, используется манипулятор `endl`, который, кроме перевода строки, сбрасывает буфер вывода.



Символ новой строки, прекращающий работу функции `getline()`, отбрасывается и в строковой переменной *не сохраняется*.

Строковые операции `size()` и `empty()`

Функция `empty()` (пусто) делает то, что и ожидается: она возвращает логическое значение `true` (раздел 2.1 стр. 61), если строка пуста, и значение `false` — в противном случае. Подобно функции-члену `isbn()` класса `Sales_item` (см. раздел 1.5.2, стр. 51), функция `empty()` является членом класса `string`. Для вызова этой функции используем точечный оператор, позволяющий указать объект, функцию `empty()` которого необходимо вызвать.

А теперь пересмотрим предыдущую программу так, чтобы она выводила только непустые строки:

```
// читать ввод построчно и отбрасывать пустые строки
while (getline(cin, line))
    if (!line.empty())
        cout << line << endl;
```

Условие использует оператор логического NOT (оператор `!`). Он возвращает инверсное значение своего операнда типа `bool`. В данном случае условие истинно, если строка `line` не пуста.

Функция `size()` возвращает длину строки (т.е. количество символов в ней). Давайте используем ее для вывода строк длиной только больше 80 символов.

```
string line;
// читать ввод построчно и отображать строки длиной более 80 символов
while (getline(cin, line))
    if (line.size() > 80)
        cout << line << endl;
```

Тип `string::size_type`

Вполне логично ожидать, что функция `size()` возвращает значение типа `int`, а учитывая сказанное в разделе 2.1.1 (стр. 64), вероятней всего, типа `unsigned`. Но вместо этого функция `size()` возвращает значение типа `string::size_type`. Этот тип требует более подробных объяснений.

В классе `string` (и нескольких других библиотечных типах) определены вспомогательные типы данных. Эти вспомогательные типы позволяют использовать библиотечные типы машинно-независимым способом. Тип `size_type` — это один из таких вспомогательных типов. Чтобы воспользоваться типом `size_type`, определенным в классе `string`, применяется оператор области видимости (оператор `::`), указывающий на то, что имя `size_type` определено в классе `string`.

Хотя точный размер типа `string::size_type` неизвестен, можно с уверенностью сказать, что этот беззнаковый тип (см. раздел 2.1.1, стр. 62) достаточно большой, чтобы содержать размер любой строки. Любая переменная, используемая для хранения результата операции `size()` класса `string`, должна иметь тип `string::size_type`.

C++ 11 По общему признанию, довольно утомительно вводить каждый раз тип `string::size_type`. По новому стандарту можно попросить компилятор самостоятельно применить соответствующий тип при помощи спецификаторов `auto` или `decltype` (см. раздел 2.5.2, стр. 107):

```
auto len = line.size(); // len имеет тип string::size_type
```

Поскольку функция `size()` возвращает беззнаковый тип, следует напомнить, что выражения, в которых смешаны знаковые и беззнаковые данные, могут дать непредвиденные результаты (см. раздел 2.1.2, стр. 67). Например, если переменная `n` типа `int` содержит отрицательное значение, то выражение `s.size() < n` почти наверняка истинно. Оно возвращает значение `true` потому, что отрицательное значение переменной `n` преобразуется в большое беззнаковое значение.



Проблем преобразования между беззнаковыми и знаковыми типами можно избежать, если не использовать переменные типа `int` в выражениях, где используется функция `size()`.

Сравнение строк

Класс string определяет несколько операторов для сравнения строк. Эти операторы сравнивают строки посимвольно. Результат сравнения зависит от регистра символов, символы в верхнем и нижнем регистре отличаются.

Операторы равенства (== и !=) проверяют, равны или не равны две строки соответственно. Две строки равны, если у них одинаковая длина и одинаковые символы. Операторы сравнения (<, >, <=, >=) проверяют, меньше ли одна строка другой, больше, меньше или равна, больше или равна другой. Эти операторы используют ту же стратегию, старшинство символов в алфавитном порядке в зависимости от регистра.

5. Если длина у двух строк разная и если каждый символ более короткой строки совпадает с соответствующим символом более длинной, то короткая строка меньше длинной.
6. Если символы в соответствующих позициях двух строк отличаются, то результат сравнения определяется первым отличающимся символом.

Для примера рассмотрим следующие строки:

```
string str = "Hello";
string phrase = "Hello World";
string slang = "Hiya";
```

Согласно правилу 1 строка str меньше строки phrase. Согласно правилу 2 строка slang больше, чем строки str и phrase.

Присвоение строк

Как правило, библиотечные типы столь же просты в применении, как и встроенные. Поэтому большинство библиотечных типов поддерживают присвоение. Строки не являются исключением, один объект класса string вполне можно присвоить другому.

```
string st1(10, 'c'), st2; // st1 - cccccccccc; st2 - пустая строка
st1 = st2; // присвоение: замена содержимого st1 копией st2
           // теперь st1 и st2 - пустые строки
```

Сложение двух строк

Результатом сложения двух строк является новая строка, объединяющая содержимое левого операнда, а затем правого. Таким образом, при применении оператора суммы (оператор +) к строкам результатом будет новая строка, символы которой являются копией символов левого операнда, сопровождаемые символами правого операнда. Составной оператор присвоения (оператор +=) (см. раздел 1.4.1, стр. 38) добавляет правый операнд к строке слева:

```
string s1 = "hello, ", s2 = "world\n";
string s3 = s1 + s2; // s3 - hello, world\n
s1 += s2;           // эквивалент to s1 = s1 + s2
```

Сложение строк и символьных строковых литералов

Как уже упоминалось в разделе 2.1.2 (стр. 66), один тип можно использовать там, где ожидается другой тип, если есть преобразование из данного типа в ожидаемый. Библиотека `string` позволяет преобразовывать как символьные, так и строковые литералы (см. раздел 2.1.3, стр. 70) в строки. Поскольку эти литералы можно использовать там, где ожидаются строки, предыдущую программу можно переписать следующим образом:

```
string s1 = "hello", s2 = "world"; // в s1 и s2 нет пунктуации
string s3 = s1 + ", " + s2 + '\n';
```

Когда объекты класса `string` смешиваются со строковыми или символьными литералами, то по крайней мере один из операндов каждого оператора `+` должен иметь тип `string`.

```
string s4 = s1 + ", ";           // ok: сложение строки и литерала
string s5 = "hello" + ", ";      // ошибка: нет строкового операнда
string s6 = s1 + ", " + "world"; // ok: каждый + имеет строковый операнд
string s7 = "hello" + ", " + s2; // ошибка: нельзя сложить строковые
                                // литералы
```

В инициализации переменных `s4` и `s5` задействовано только по одному оператору, поэтому достаточно просто проверить его корректность. Инициализация переменной `s6` может показаться странной, но работает она аналогично объединенным в цепочку операторам ввода или вывода (см. раздел 1.2 стр. 30). Это эквивалентно следующему коду:

```
string s6 = (s1 + ", ") + "world";
```

Часть `s1 + ", "` выражения возвращает объект класса `string`, она составляет левый операнд второго оператора `+`. Это эквивалентно следующему коду:

```
string tmp = s1 + ", "; // ok: + имеет строковый операнд
s6 = tmp + "world";      // ok: + имеет строковый операнд
```

С другой стороны, инициализация переменной `s7` недопустима, и это становится очевидным, если заключить часть выражения в скобки:

```
string s7 = ("hello" + ", ") + s2; // ошибка: нельзя сложить строковые
                                // литералы
```

Теперь довольно просто заметить, что первая часть выражения суммирует два строковых литерала. Поскольку это невозможно, оператор недопустим.



ВНИМАНИЕ

По историческим причинам и для совместимости с языком C строковые литералы *не принадлежат* к типу `string` стандартной библиотеки. При использовании строковых литералов и библиотечного типа `string`, не следует забывать, что это разные типы.

Упражнения раздела 3.2.2

Упражнение 3.2. Напишите программу, читающую со стандартного устройства ввода по одной строке за раз. Измените программу так, чтобы читать по одному слову за раз.

Упражнение 3.3. Объясните, как символы пробелов обрабатываются в операторе ввода класса `string` и в функции `getline()`.

Упражнение 3.4. Напишите программу, читающую две строки и сообщающую, равны ли они. В противном случае программа сообщает, которая из них больше. Затем измените программу так, чтобы она сообщала, одинаковая ли у строк длина, а в противном случае — которая из них длиннее.

Упражнение 3.5. Напишите программу, читающую строки со стандартного устройства ввода и суммирующую их в одну большую строку. Отобразите полученную строку. Затем измените программу так, чтобы отделять соседние введенные строки пробелами.



3.2.3. Работа с символами строки

Зачастую приходится работать с индивидуальными символами строки. Например, может понадобиться выяснить, является ли определенный символ пробелом, или изменить регистр символов на нижний, или узнать, присутствует ли некий символ в строке, и т.д.

Одной из частей этих действий является доступ к самим символам строки. Иногда необходима обработка каждого символа, а иногда лишь определенного символа, либо может понадобиться прекратить обработку, как только выполнится некое условие. Кроме того, это наилучший способ справиться со случаями, когда задействуются разные языковые и библиотечные средства.

Другой частью обработки символов является выяснение и (или) изменение их характеристик. Эта часть задачи выполняется набором библиотечных функций, описанных в табл. 3.3. Данные функции определены в заголовке `cctype`.

Таблица 3.3. Функции `cctype`

<code>isalnum(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> является буквой или цифрой
<code>isalpha(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — буква
<code>iscntrl(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — управляющий символ
<code>isdigit(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — цифра
<code>isgraph(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — не пробел, а печатаемый символ
<code>islower(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — символ в нижнем регистре

Окончание табл. 3.3

<code>isprint(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — печатаемый символ
<code>ispunct(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — знак пунктуации (т.е. символ, который не является управляющим символом, цифрой, символом или печатаемым отступом)
<code>isspace(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — символ отступа (т.е. пробел, табуляция, вертикальная табуляция, возврат, новая строка или прогон страницы)
<code>isupper(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — символ в верхнем регистре
<code>isxdigit(c)</code>	Возвращает значение <code>true</code> , если <code>c</code> — шестнадцатеричная цифра
<code>tolower(c)</code>	Если <code>c</code> — прописная буква, возвращает ее эквивалент в нижнем регистре, в противном случае возвращает символ <code>c</code> неизменным
<code>toupper(c)</code>	Если <code>c</code> — строчная буква, возвращает ее эквивалент в верхнем регистре, в противном случае возвращает символ <code>c</code> неизменным

СОВЕТ. ИСПОЛЬЗУЙТЕ ВЕРСИИ C++ БИБЛИОТЕЧНЫХ ЗАГОЛОВКОВ ЯЗЫКА C

Кроме средств, определенных специально для языка C++, его библиотека содержит также библиотеку языка C. Имена заголовков языка C имеют формат *имя.h*. Версии этих же заголовков языка C++ имеют формат *сима*, т.е. суффикс `.h` удален, а *имени* предшествует символ `c`, означающий, что этот заголовок принадлежит библиотеке C.

Следовательно, у заголовка `cctype` то же содержимое, что и у заголовка `cctype.h`, но в форме, соответствующей программе C++. В частности, имена, определенные в заголовках *сима*, определены также в пространстве имен `std`, тогда как имена, определенные в заголовках `.h`, — нет.

Как правило, в программах на языке C++ используют заголовки версии *сима*, а не *имя.h*. Таким образом, имена из стандартной библиотеки будут быстро найдены в пространстве имен `std`. Использование заголовка `.h` возлагает на программиста дополнительную заботу по отслеживанию, какие из библиотечных имен унаследованы от языка C, а какие принадлежат языку C++.

Обработка каждого символа, использование серийного оператора `for`

C++ 11 Если необходимо сделать нечто с каждым символом в строке, то наилучшим подходом является использование оператора, введенного новым стандартом, — *серийный оператор* `for (range for)`. Этот оператор перебирает элементы данной ему последовательности и выполняет с каждым из них некую операцию. Его синтаксическая форма такова:

```
for (объявление : выражение)
    оператор
```

где *выражение* — это объект типа, который представляет последовательность, а *объявление* определяет переменную, которая будет использована для доступа к элементам последовательности. На каждой итерации переменная в *объявлении* инициализируется значением следующего элемента в *выражении*.

Строка представляет собой последовательность символов, поэтому объект типа string можно использовать как *выражение* в серийном операторе for. Например, серийный оператор for можно использовать для вывода каждого символа строки в отдельной строке вывода.

```
string str("some string");
// вывести символы строки str по одному на строку
for (auto c : str)    // для каждого символа в строке str
    cout << c << endl; // вывести текущий символ и символ новой строки
```

Цикл for ассоциирует переменную c с переменной str типа string. Управляющая переменная цикла определяется тем же способом, что и любая другая переменная. В данном случае используется спецификатор auto (см. раздел 2.5.2, стр. 107), чтобы позволить компилятору самостоятельно определять тип переменной c, которым в данном случае будет тип char. На каждой итерации следующий символ строки str будет скопирован в переменную c. Таким образом, можно прочитать этот цикл так: “Для каждого символа c в строке str” сделать нечто. Под “нечто” в данном случае подразумевается вывод текущего символа, сопровождаемого символом новой строки.

Рассмотрим более сложный пример и используем серийный оператор for, а также функцию ispunct() для подсчета количества знаков пунктуации в строке:

```
string s("Hello World!!!");
// punct_cnt имеет тот же тип, что и у возвращаемого значения
// функции s.size(); см. р. 2.5.3 (стр. 109)
decltype(s.size()) punct_cnt = 0;
// подсчитать количество знаков пунктуации в строке s
for (auto c : s)    // для каждого символа в строке s
    if (ispunct(c)) // если символ знак пунктуации
        ++punct_cnt; // увеличить счетчик пунктуаций
cout << punct_cnt
    << " punctuation characters in " << s << endl;
```

Вывод этой программы таков:

```
3 punctuation characters in Hello World!!!
```

Здесь для объявления счетчика punct_cnt используется спецификатор decltype (см. раздел 2.5.3, стр. 110). Его тип совпадает с типом возвращаемого значения функции s.size(), которым является тип string::size_type. Для обработки каждого символа в строке используем серийный оператор for. На сей раз проверяется, является ли каждый символ знаком пунктуации. Если да, то используем оператор инкремента (см. раздел 1.4.1, стр. 38) для добавле-

ния единицы к счетчику. Когда серийный оператор `for` завершает работу, отображается результат.

Использование серийного оператора `for` для изменения символов в строке

Если необходимо изменить значение символов в строке, переменную цикла следует определить как ссылочный тип (см. раздел 2.3.1, стр. 85). Помните, что ссылка — это только другое имя для данного объекта. При использовании ссылки в качестве управляющей переменной она будет по очереди связана с каждым элементом последовательности. Используя ссылку, можно изменить символ, с которым она связана.

Предположим, что вместо подсчета знаков пунктуации необходимо преобразовать все буквы строки в верхний регистр. Для этого можно использовать библиотечную функцию `toupper()`, которая возвращает полученный символ в верхнем регистре. Для преобразования всей строки необходимо вызвать функцию `toupper()` для каждого символа и записать результат в тот же символ:

```
string s("Hello World!!!");  
// преобразовать s в верхний регистр  
for (auto &c : s)    // для каждого символа в строке s  
                    // (примечание: c - ссылка)  
    c = toupper(c); // c - ссылка, поэтому присвоение изменяет  
                    // символ в строке s  
cout << s << endl;
```

Вывод этого кода таков:

```
HELLO WORLD!!!
```

На каждой итерации переменная `c` ссылается на следующий символ строки `s`. При присвоении значения переменной `c` изменяется соответствующий символ в строке `s`.

```
c = toupper(c); // c - ссылка, поэтому присвоение изменяет  
                // символ в строке s
```

Таким образом, данное выражение изменяет значение символа, с которым связана переменная `c`. По завершении этого цикла все символы в строке `str` будут в верхнем регистре.

Обработка лишь некоторых символов

Серийный оператор `for` работает хорошо, когда необходимо обработать каждый символ. Но иногда необходим доступ только к одному символу или к некоторому количеству символов на основании некоего условия. Например, можно преобразовать в верхний регистр только первый символ строки или только первое слово в строке.

Существуют два способа доступа к отдельным символам в строке: можно использовать индексирование или итератор. Более подробная информация об итераторах приведена в разделе 3.4 (стр. 153) и в главе 9.

Оператор индексирования (оператор `[]`) получает значение типа `string::size_type` (раздел 3.2.2, стр. 131), обозначающее позицию символа, к которому необходим доступ. Оператор возвращает ссылку на символ в указанной позиции.

Индексация строк начинается с нуля; если строка `s` содержит по крайней мере два символа, то первым будет символ `s[0]`, вторым — `s[1]`, а последним символом является `s[s.size() - 1]`.



Значения, используемые для индексирования строк, не должны быть отрицательными и не должны превосходить размер строки (≥ 0 и $< \text{size}()$). Результат использования индекса вне этого диапазона непредсказуем. Непредсказуема также индексация пустой строки.

Значение оператора индексирования называется *индексом* (index). Индекс может быть любым выражением, возвращающим целочисленное значение. Если у индекса будет знаковый тип, то его значение преобразуется в беззнаковый тип `size_type` (см. раздел 2.1.2, стр. 67).

Следующий пример использует оператор индексирования для вывода первого символа строки:

```
if (!s.empty())           // удостоверившись, что символ для вывода есть,
    cout << s[0] << endl; // вывести первый символ строки s
```

Прежде чем обратиться к символу, удостоверимся, что строка `s` не пуста. При каждом использовании индексирования следует проверять наличие значения в данной области. Если строка `s` пуста, то значение `s[0]` неопределенно.

Если строка не константа (см. раздел 2.4, стр. 95), возвращенному оператором индексирования символу можно присвоить новое значение. Например, первый символ можно перевести в верхний регистр следующим образом:

```
string s("some string");
if (!s.empty())           // удостовериться в наличии символа s[0]
    s[0] = toupper(s[0]); // присвоить новое значение первому символу
```

Вывод этой программы приведен ниже.

Some string

Использование индексирования для перебора

В следующем примере переведем в верхний регистр первое слово строки `s`:

```
// обрабатывать символы строки s, пока они не исчерпаются или
// не встретится пробел
```

```
for (decltype(s.size()) index = 0;  
    index != s.size() && !isspace(s[index]); ++index)  
    s[index] = toupper(s[index]); // преобразовать в верхний регистр
```

Вывод этой программы таков:

SOME string

Цикл `for` (см. раздел 1.4.2, стр. 39) использует переменную `index` для индексирования строки `s`. Для присвоения переменной `index` соответствующего типа используется спецификатор `decltype`. Переменную `index` инициализируем значением 0, чтобы первая итерация началась с первого символа строки `s`. На каждой итерации значение переменной `index` увеличивается, чтобы получить следующий символ строки `s`. В теле цикла текущий символ переводится в верхний регистр.

В условии цикла `for` используется новая часть — оператор логического AND (оператор `&&`). Этот оператор возвращает значение `true`, если оба операнда истинны, и значение `false` в противном случае. Важно то, что этот оператор гарантирует обработку своего правого операнда, *только если* левый операнд истинен. В данном случае это гарантирует, что индексирования строки `s` не будет, если переменная `index` находится вне диапазона. Таким образом, часть `s[index]` выполняется, только если переменная `index` не равна `s.size()`. Поскольку инкремент переменной `index` никогда не превзойдет значения `s.size()`, переменная `index` всегда будет меньше `s.size()`.

ВНИМАНИЕ! ИНДЕКСИРОВАНИЕ НЕ КОНТРОЛИРУЕТСЯ

При использовании индексирования следует самому позаботиться о том, чтобы индекс оставался в допустимом диапазоне. Индекс должен быть ≥ 0 и $< \text{size}()$ строки. Для упрощения кода, использующего индексирование, в качестве индекса *всегда* следует использовать переменную типа `string::size_type`. Поскольку это беззнаковый тип, индекс не может быть меньше нуля. При использовании значения типа `size_type` в качестве индекса достаточно проверять только то, что значение индекса меньше значения, возвращаемого функцией `size()`.



ВНИМАНИЕ

Библиотека не обязана проверять и не проверяет значение индекса. Результат использования индекса вне диапазона непредсказуем.

Использование индексирования для произвольного доступа

В предыдущем примере преобразования регистра символов последовательности индекс перемещался на одну позицию за раз. Но можно также вычислить индекс и непосредственно обратиться к выбранному символу. Нет никакой необходимости получать доступ к символам последовательно.

Предположим, например, что имеется число от 0 до 15, которое необходимо представить в шестнадцатеричном виде. Для этого можно использовать строку, инициализированную шестнадцатью шестнадцатеричными цифрами.

```
const string hexdigits = "0123456789ABCDEF"; // возможные
                                                // шестнадцатеричные цифры
cout << "Enter a series of numbers between 0 and 15"
    << " separated by spaces. Hit ENTER when finished: "
    << endl;
string result; // будет содержать результирующую
               // шестнадцатеричную строку
string::size_type n; // содержит введенное число
while (cin >> n)
    if (n < hexdigits.size()) // игнорировать недопустимый ввод
        result += hexdigits[n]; // выбрать указанную
                                // шестнадцатеричную цифру
cout << "Your hex number is: " << result << endl;
```

Если ввести следующие числа:

12 0 5 15 8 15

то результат будет таким:

Your hex number is: C05F8F

Программа начинается с инициализации строки `hexdigits`, содержащей шестнадцатеричные цифры от 0 до F. Сделаем эту строку константной (см. раздел 2.4, стр. 95), поскольку содержащиеся в ней значения не должны изменяться. Для индексирования строки `hexdigits` используем в цикле введенное значение `n`. Значением `hexdigits[n]` является символ, расположенный в позиции `n` строки `hexdigits`. Например, если `n` равно 15, то результат — F; если 12, то результат — C и т.д. Полученная цифра добавляется к переменной `result`, которая и выводится, когда весь ввод прочитан.

Всякий раз, когда используется индексирование, следует позаботиться о том, чтобы индекс оставался в диапазоне. В этой программе индекс, `n`, имеет тип `string::size_type`, который, как известно, является беззнаковым. В результате значение переменной `n` гарантированно будет больше или равно 0. Прежде чем использовать переменную `n` для индексирования строки `hexdigits`, удостоверимся, что ее значение меньше, чем `hexdigits.size()`.

Упражнения раздела 3.2.3

Упражнение 3.6. Используйте серийный оператор `for` для замены всех символов строки на X.

Упражнение 3.7. Что будет, если определить управляющую переменную цикла в предыдущем упражнении как имеющую тип `char`? Предскажите

результат, а затем измените программу так, чтобы использовался тип `char`, и убедитесь в своей правоте.

Упражнение 3.8. Перепишите программу первого упражнения, сначала используя оператор `while`, а затем традиционный цикл `for`. Какой из этих трех подходов вы предпочтете и почему?

Упражнение 3.9. Что делает следующая программа? Действительно ли она корректна? Если нет, то почему?

```
string s;
cout << s[0] << endl;
```

Упражнение 3.10. Напишите программу, которая читает строку символов, включающую знаки пунктуации, и выведите ее, но уже без знаков пунктуации.

Упражнение 3.11. Допустим ли следующий серийный оператор `for`? Если да, то каков тип переменной `c`?

```
const string s = "Keep out!";
for (auto &c : s) { /* ... */ }
```



3.3. Библиотечный тип `vector`

Вектор (`vector`) — это коллекция объектов одинакового типа, каждому из которых присвоен целочисленный индекс, предоставляющий доступ к этому объекту. Вектор — это *контейнер* (`container`), поскольку он “содержит” другие объекты. Более подробная информация о контейнерах приведена в части II.

Чтобы использовать вектор, необходимо включить соответствующий заголовок. В примерах подразумевается также, что включено соответствующее объявление `using`.

```
#include <vector>
using std::vector;
```

Тип `vector` — это *шаблон класса* (`class template`). Язык C++ поддерживают шаблоны и классов, и функций. Написание шаблона требует довольно глубокого понимания языка C++. До главы 16 мы даже не будем рассматривать создание собственных шаблонов! К счастью, чтобы использовать шаблоны, вовсе не обязательно уметь их создавать.

Шаблоны сами по себе не являются ни функциями, ни классами. Их можно считать инструкцией для компилятора по созданию классов или функций. Процесс, используемый компилятором для создания классов или функций по шаблону, называется *созданием экземпляра* (`instantiation`) шаблона. При использовании шаблона необходимо указать, экземпляр какого класса или функции должен создать компилятор.

Для создания экземпляра шаблона класса следует указать дополнительную информацию, характер которой зависит от шаблона. Эта информация всегда задается одинаково: в угловых скобках после имени шаблона.

В случае вектора предоставляемой дополнительной информацией является тип объектов, которые он должен содержать:

```
vector<int> ivec;           // ivec содержит объекты типа int
vector<Sales_item> Sales_vec; // содержит объекты класса Sales_item
vector<vector<string>> file; // вектор, содержащий другие векторы
```

В этом примере компилятор создает три разных экземпляра шаблона vector: vector<int>, vector<Sales_item> и vector<vector<string>>.



vector — это шаблон, а не класс. Классам, созданным по шаблону vector, следует указать тип хранимого элемента, например vector<int>.

Можно определить векторы для содержания объектов практически любого типа. Поскольку ссылки не объекты (см. раздел 2.3.1 стр. 85), не может быть вектора ссылок. Однако векторы большинства других (не ссылочных) встроенных типов и типов классов вполне могут существовать. В частности, может быть вектор, элементами которого являются другие векторы.



Следует заметить, что прежние версии языка C++ использовали несколько иной синтаксис определения вектора, элементы которого сами являлись экземплярами шаблона vector (или другого типа шаблона). Прежде необходимо было ставить пробел между закрывающей угловой скобкой внешней части vector и типом его элемента: т.е. vector<vector<int> >, а не vector<vector<int>>.



Некоторые компиляторы могут потребовать объявления вектора векторов в старом стиле, например vector<vector<int> >.



3.3.1. Определение и инициализация векторов

Подобно любому типу класса, шаблон vector контролирует способ определения и инициализации векторов. Наиболее распространенные способы определения векторов приведены в табл. 3.4.

Инициализация вектора по умолчанию (см. раздел 2.2.1, стр. 77) позволяет создать пустой вектор определенного типа:

```
vector<string> svec; // инициализация по умолчанию; у svec нет элементов
```

Могло бы показаться, что пустой вектор бесполезен. Однако, как будет продемонстрировано вскоре, элементы в вектор можно без проблем добавлять и во время выполнения. В действительности наиболее распространенный спо-

с об использования векторов подразумевает определение первоначально пустого вектора, в который элементы добавляются по мере необходимости во время выполнения.

Таблица 3.4. Способы инициализации векторов

<code>vector<T> v1</code>	Вектор, содержащий объекты типа <code>T</code> . Стандартный конструктор <code>v1</code> пуст
<code>vector<T> v2(v1)</code>	Вектор <code>v2</code> — копия всех элементов вектора <code>v1</code>
<code>vector<T> v2 = v1</code>	Эквивалент <code>v2(v1)</code> , <code>v2</code> — копия элементов вектора <code>v1</code>
<code>vector<T> v3(n, val)</code>	Вектор <code>v3</code> содержит <code>n</code> элементов со значением <code>val</code>
<code>vector<T> v4(n)</code>	Вектор <code>v4</code> содержит <code>n</code> экземпляров объекта типа <code>T</code> , инициализированного значением по умолчанию
<code>vector<T> v5{a,b,c ...}</code>	Вектор <code>v5</code> содержит столько элементов, сколько предоставлено инициализаторов; элементы инициализируются соответствующими инициализаторами
<code>vector<T> v5 = {a,b,c ... }</code>	Эквивалент <code>v5{a,b,c ... }</code>

При определении вектора для его элементов можно также предоставить исходное значение (или значения). Например, можно скопировать элементы из другого вектора. При копировании векторов каждый элемент нового вектора будет копией соответствующего элемента исходного. Оба вектора должны иметь тот же тип:

```
vector<int> ivec;           // первоначально пустой
// присвоить ivec несколько значений
vector<int> ivec2(ivec);    // копировать элементы ivec в ivec2
vector<int> ivec3 = ivec;    // копировать элементы ivec в ivec3
vector<string> svec(ivec2); // svec содержит строки,
                           // а не целые числа
```

Списочная инициализация вектора

C++ 11 Согласно новому стандарту, еще одним способом предоставления значений элементам вектора является списочная инициализация (см. раздел 2.2.1, стр. 77), т.е. заключенный в фигурные скобки список любого количества начальных значений элементов:

```
vector<string> articles = {"a", "an", "the"};
```

В результате у вектора будет три элемента: первый со значением "a", второй — "an", последний — "the".

Как уже упоминалось, язык C++ предоставляет несколько форм инициализации (см. раздел 2.2.1, стр. 77). Во многих, но не во всех случаях эти формы

инициализации можно использовать взаимозаменяемо. На настоящий момент приводились примеры двух форм инициализации: инициализация копией (с использованием знака =) (см. раздел 3.2.1, стр. 126), когда предоставляется только один инициализатор; и внутрикласовая инициализация (см. раздел 2.6.1, стр. 113). Третий способ подразумевает предоставление списка значений элементов, заключенных в фигурные скобки (списочная инициализация). Нельзя предоставить список инициализаторов, используя круглые скобки.

```
vector<string> v1{"a", "an", "the"}; // списочная инициализация
vector<string> v2("a", "an", "the"); // ошибка
```

Создание определенного количества элементов

Вектор можно также инициализировать набором из определенного количества элементов, обладающих указанным значением. Счетчик задает количество элементов, а за ним следует исходное значение для каждого из этих элементов.

```
vector<int> ivec(10, -1); // десять элементов типа int, каждый из
                          // которых инициализирован значением -1
vector<string> svec(10, "hi!"); // десять строк, инициализированных
                               // значением "hi!"
```

Инициализация значения

Иногда инициализирующее значение можно пропустить и указать только размер. В этом случае произойдет *инициализация значения* (value initialization), т.е. библиотека создаст инициализатор элемента сама. Это созданное библиотекой значение используется для инициализации каждого элемента в контейнере. Значение инициализатора элемента вектора зависит от типа его элементов.

Если вектор хранит элементы встроенного типа, такие как `int`, то инициализатором элемента будет значение 0. Если элементы имеют тип класса, такой как `string`, то инициализатором элемента будет его значение по умолчанию.

```
vector<int> ivec(10); // десять элементов, инициализированных
                     // значением 0
vector<string> svec(10); // десять элементов, инициализированных
                        // пустой строкой
```

Эта форма инициализации имеет два ограничения. Первое — некоторые классы всегда требуют явного предоставления инициализатора (см. раздел 2.2.1, стр. 77). Если вектор содержит объекты, тип которых не имеет значения по умолчанию, то начальное значение элемента следует предоставить самому; невозможно создать векторы таких типов, предоставив только размер.

Второе ограничение заключается в том, что при предоставлении количества элементов без исходного значения необходимо использовать *прямую инициализацию* (direct initialization):

```
vector<int> vi = 10; // ошибка: необходима прямая инициализация
```

Здесь число 10 используется для указания на то, как создать вектор, — необходимо, чтобы он обладал десятью элементами с инициализированными значениями. Число 10 не “копируется” в вектор. Следовательно, нельзя использовать форму инициализации копией. Более подробная информация об этом ограничении приведена в разделе 7.5.4 (стр. 382).



Списочный инициализатор или количество элементов

В некоторых случаях смысл инициализации зависит от того, используются ли при передаче инициализаторов фигурные скобки или круглые. Например, при инициализации вектора `vector<int>` одиночным целочисленным значением это значение могло бы означать либо размер вектора, либо значение элемента. Точно так же, если предоставить два целочисленных значения, то они могли бы быть размером и исходным значением или значениями для двух элементов вектора. Для определения предназначения используются фигурные или круглые скобки.

```
vector<int> v1(10);    // v1 имеет десять элементов со значением 0
vector<int> v2{10};   // v2 имеет один элемент со значением 10
vector<int> v3(10, 1); // v3 имеет десять элементов со значением 1
vector<int> v4{10, 1}; // v4 имеет два элемента со значениями 10 и 1
```

Круглые скобки позволяют сообщить, что предоставленные значения должны использоваться для *создания* объекта. Таким образом, векторы `v1` и `v3` используют свои инициализаторы для определения размера вектора, а также размера и значения его элементов соответственно.

Использование фигурных скобок, `{...}`, означает попытку списочной инициализации. Таким образом, если класс способен использовать значения в фигурных скобках как список инициализаторов элементов, то он так и сделает. Если это невозможно, то следует рассмотреть другие способы инициализации объектов. Значения, предоставленные при инициализации векторов `v2` и `v4`, рассматриваются как значения элементов. Это списочная инициализация объектов; у полученных векторов будет один и два элемента соответственно.

С другой стороны, если используются фигурные скобки и нет никакой возможности использовать инициализаторы для списочной инициализации объектов, то эти значения будут использоваться для создания объектов. Например, для списочной инициализации вектора строк следует предоставлять значения, которые можно рассматривать как строки. В данном случае нет никаких сомнений, осуществляется ли списочная инициализация элементов или создание вектора указанного размера.

```
vector<string> v5{"hi"}; // списочная инициализация: v5 имеет
                        // один элемент
vector<string> v6("hi"); // ошибка: нельзя создать вектор из
                        // строкового литерала
vector<string> v7{10};   // v7 имеет десять элементов, инициализированных
```

```

// значением по умолчанию
vector<string> v8{10, "hi"}; // v8 имеет десять элементов со
// значением "hi"

```

Хотя фигурные скобки использованы во всех этих определениях, кроме одного, только вектор `v5` имеет списочную инициализацию. Для списочной инициализации вектора значения в фигурных скобках должны соответствовать типу элемента. Нельзя использовать объект типа `int` для инициализации строки, поэтому инициализаторы векторов `v7` и `v8` не могут быть инициализаторами элементов. Если списочная инициализация невозможна, компилятор ищет другие способы инициализации объектов.

Упражнения раздела 3.3.1

Упражнение 3.12. Есть ли ошибки в следующих определениях векторов? Объясните, что делают допустимые определения. Объясните, почему некорректны недопустимые определения.

- (a) `vector<vector<int>> ivec;`
- (b) `vector<string> svec = ivec;`
- (c) `vector<string> svec(10, "null");`

Упражнение 3.13. Сколько элементов находится в каждом из следующих векторов? Каковы значения этих элементов?

- (a) `vector<int> v1;`
- (b) `vector<int> v2{10};`
- (c) `vector<int> v3{10, 42};`
- (d) `vector<int> v4{10};`
- (e) `vector<int> v5{10, 42};`
- (f) `vector<string> v6{10};`
- (g) `vector<string> v7{10, "hi"};`



3.3.2. Добавление элементов в вектор

Прямая инициализация элементов вектора осуществима только при небольшом количестве исходных значений, при копировании другого вектора и при инициализации всех элементов тем же значением. Но обычно при создании вектора неизвестно ни количество его элементов, ни их значения. Но даже если все значения известны, то определение большого количества разных начальных значений может оказаться очень громоздким, чтобы располагать его в месте создания вектора.

Если необходим вектор со значениями от 0 до 9, то можно легко использовать списочную инициализацию. Но что если необходимы элементы от 0 до 99 или от 0 до 999? Списочная инициализация была бы слишком громоздкой. В таких случаях лучше создать пустой вектор и использовать его функцию-член `push_back()`, чтобы добавить элементы во время выполнения. Функция `push_back()` вставляет переданное ей значение в вектор как новый последний элемент. Рассмотрим пример.

```
vector<int> v2;           // пустой вектор
for (int i = 0; i != 100; ++i)
    v2.push_back(i); // добавить последовательность целых чисел в v2
// по завершении цикла v2 имеет 100 элементов со значениями от 0 до 99
```

Хотя заранее известно, что будет 100 элементов, вектор `v2` определяется как пустой. Каждая итерация добавляет следующее по порядку целое число в вектор `v2` как новый элемент.

Тот же подход используется, если необходимо создать вектор, количество элементов которого до времени выполнения неизвестно. Например, в вектор можно читать введенные пользователем значения.

```
// читать слова со стандартного устройства ввода и сохранять их
// в векторе как элементы
string word;
vector<string> text;           // пустой вектор
while (cin >> word) {
    text.push_back(word); // добавить слово в текст
}
```

И снова все начинается с пустого вектора. На сей раз, неизвестное количество значений читается и сохраняется в векторе строк `text`.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. РОСТ ВЕКТОРА ЭФФЕКТИВЕН

Стандарт требует, чтобы реализация шаблона `vector` обеспечивала эффективное добавление элементов во время выполнения. Поскольку рост вектора эффективен, определение вектора сразу необходимого размера зачастую является ненужным и может даже привести к потере производительности. Исключением является случай, когда все элементы нуждаются в одинаковом значении. При разных значениях элементов обычно эффективней определить пустой вектор и добавлять элементы во время выполнения, по мере того, как значения становятся известны. Кроме того, как будет продемонстрировано в разделе 9.4 (стр. 457), шаблон `vector` предоставляет возможности для дальнейшего увеличения производительности при добавлении элементов во время выполнения.

Начало с пустого вектора и добавление элементов во время выполнения кардинально отличается от использования встроенных массивов в языке C и других языках. В частности, если вы знакомы с языком C или Java, то, вероятно, полагаете, что лучше определить вектор в его ожидаемом размере, но фактически имеет место обратное.

Последствия возможности добавления элементов в вектор

Тот факт, что добавление элементов в вектор весьма эффективно, существенно упрощает многие задачи программирования. Но эта простота налагает

новые обязательства на наши программы: необходимо гарантировать корректность всех циклов, даже если цикл изменяет размер вектора.

Другое последствие динамического характера векторов станет яснее, когда мы узнаем больше об их использовании. Но есть одно последствие, на которое стоит обратить внимание уже сейчас: по причинам, изложенным в разделе 5.4.3 (стр. 252), нельзя использовать серийный оператор `for`, если тело цикла добавляет элементы в вектор.



ВНИМАНИЕ

Тело серийного оператора `for` не должно изменять размер перебираемой последовательности.

Упражнения раздела 3.3.2

Упражнение 3.14. Напишите программу, читающую последовательность целых чисел из потока `cin` и сохраняющую их в векторе.

Упражнение 3.15. Повторите предыдущую программу, но на сей раз читайте строки.



3.3.3. Другие операции с векторами

Кроме функции `push_back()`, шаблон `vector` предоставляет еще несколько операций, большинство из которых подобно соответствующим операциям класса `string`. Наиболее важные из них приведены в табл. 3.5.

Таблица 3.5. Операции с векторами

<code>v.empty()</code>	Возвращает значение <code>true</code> , если вектор <code>v</code> пуст. В противном случае возвращает значение <code>false</code>
<code>v.size()</code>	Возвращает количество элементов вектора <code>v</code>
<code>v.push_back(t)</code>	Добавляет элемент со значением <code>t</code> в конец вектора <code>v</code>
<code>v[n]</code>	Возвращает ссылку на элемент в позиции <code>n</code> вектора <code>v</code>
<code>v1 = v2</code>	Заменяет элементы вектора <code>v1</code> копией элементов вектора <code>v2</code>
<code>v1 = {a,b,c ... }</code>	Заменяет элементы вектора <code>v1</code> копией элементов из разделяемого запятыми списка
<code>v1 == v2</code> <code>v1 != v2</code>	Векторы <code>v1</code> и <code>v2</code> равны, если они содержат одинаковые элементы в тех же позициях
<code><, <=, >, >=</code>	Имеют обычное значение и полагаются на алфавитный порядок

Доступ к элементам вектора осуществляется таким же способом, как и к символам строки: по их позиции в векторе. Например, для обработки все элементов вектора можно использовать серийный оператор `for` (раздел 3.2.3, стр. 134).

```
vector<int> v{1,2,3,4,5,6,7,8,9};
for (auto &i : v)           // для каждого элемента вектора v
                           // (обратите внимание: i - ссылка)
    i *= i;                // квадрат значения элемента
for (auto i : v)           // для каждого элемента вектора v
    cout << i << " ";    // вывод элемента
cout << endl;
```

В первом цикле управляющая переменная `i` определяется как ссылка, чтобы использовать ее для присвоения новых значений элементам вектора `v`. Используя спецификатор `auto`, позволим вывести ее тип автоматически. Этот цикл использует новую форму составного оператора присвоения (раздел 1.4.1, стр. 38). Как известно, оператор `+=` добавляет правый операнд к левому и сохраняет результат в левом операнде. Оператор `*=` ведет себя точно так же, но перемножает левый и правый операнды, сохраняя результат в левом операнде. Второй серийный оператор `for` отображает каждый элемент.

Функции-члены `empty()` и `size()` вектора ведут себя так же, как и соответствующие функции класса `string` (раздел 3.2.2, стр. 130): функция `empty()` возвращает логическое значение, указывающее, содержит ли вектор какие-нибудь элементы, а функция `size()` возвращает их количество. Функция-член `size()` возвращает значение типа `size_type`, определенное соответствующим типом шаблона `vector`.



Чтобы использовать тип `size_type`, необходимо указать тип, для которого он определен. Для типа `vector` *всегда* необходимо указывать тип хранимого элемента (раздел 3.3, стр. 141).

```
vector<int>::size_type // ok
vector::size_type      // ошибка
```

Операторы равенства и сравнения вектора ведут себя как соответствующие операторы класса `string` (раздел 3.2.2 стр. 131). Два вектора равны, если у них одинаковое количество элементов и значения соответствующих элементов совпадают. Операторы сравнения полагаются на алфавитный порядок: если у векторов разные размеры, но соответствующие элементы равны, то вектор с меньшим количеством элементов меньше вектора с большим количеством элементов. Если у элементов векторов разные значения, то их отношения определяются по первым отличающимся элементам.

Сравнить два вектора можно только в том случае, если возможно сравнить элементы этих векторов. Некоторые классы, такие как `string`, определяют смысл операторов равенства и сравнения. Другие, такие как класс `Sales_item`, этого не делают. Операции, поддерживаемые классом `Sales_item`, перечисле-

ны в разделе 1.5.1 (стр. 47). Они не включают ни операторов равенства, ни сравнения. В результате нельзя сравнить два вектора объектов класса `Sales_item`.

Вычисление индекса вектора

Используя оператор индексирования (раздел 3.2.3, стр. 138), можно выбрать указанный элемент. Подобно строкам, индексирование вектора начинается с 0; индекс имеет тип `size_type` соответствующего типа; и если вектор не константен, то в возвращенный оператором индексирования элемент можно осуществить запись. Кроме того, как было продемонстрировано в разделе 3.2.3 (стр. 139), можно вычислить индекс и непосредственно обратиться к элементу в данной позиции.

Предположим, имеется набор оценок степеней в диапазоне от 0 до 100. Необходимо рассчитать, сколько оценок попадает в кластер по 10. Между нулем и 100 возможна 101 оценка. Эти оценки могут быть представлены 11 кластерами: 10 кластеров по 10 оценок каждый плюс один кластер для наивысшей оценки 100. Первый кластер подсчитывает оценки от 0 до 9, второй — от 10 до 19 и т.д. Заключительный кластер подсчитывает количество оценок 100.

Таким образом, если введены следующие оценки:

42 65 95 100 39 67 95 76 88 76 83 92 76 93

результат их кластеризации должен быть таким:

0 0 0 1 1 0 2 3 2 4 1

Он означает, что не было никаких оценок ниже 30, одна оценка в 30-х, одна в 40-х, ни одной в 50-х, две в 60-х, три в 70-х, две в 80-х, четыре в 90-х и одна оценка 100.

Используем для содержания счетчиков каждого кластера вектор с 11 элементами. Индекс кластера для данной оценки можно определить делением этой оценки на 10. При делении двух целых чисел получается целое число, дробная часть которого усекается. Например, $42/10=4$, $65/10=6$, а $100/10=10$. Как только индекс кластера будет вычислен, его можно использовать для индексирования вектора и доступа к счетчику, значение которого необходимо увеличить.

```
// подсчет количества оценок в кластере по десять: 0--9,
// 10--19, ... 90--99, 100
vector<unsigned> scores(11, 0); // 11 ячеек, все со значением 0
unsigned grade;
while (cin >> grade) {           // читать оценки
    if (grade <= 100)             // обрабатывать только допустимые оценки
        ++scores[grade/10];      // приращение счетчика текущего кластера
}
```


Код начинается с определения вектора для хранения счетчиков кластеров. В данном случае все элементы должны иметь одинаковое значение, поэтому резервируем 11 элементов, каждый из которых инициализируем значением 0. Условие цикла `while` читает оценки. В цикле проверяется допустимость значения прочитанной оценки (т.е. оно меньше или равно 100). Если оценка допустима, то увеличиваем соответствующий счетчик.

Оператор, осуществляющий приращение, является хорошим примером краткости кода C++:

```
++scores[grade/10]; // приращение счетчика текущего кластера
```

Это выражение эквивалентно следующему:

```
auto ind = grade/10;           // получить индекс ячейки
scores[ind] = scores[ind] + 1; // приращение счетчика
```

Индекс ячейки вычисляется делением значения переменной `grade` на 10. Полученный результат используется для индексирования вектора `scores`, что обеспечивает доступ к соответствующему счетчику для этой оценки. Увеличение значения этого элемента означает принадлежность текущей оценки данному диапазону.

Как уже упоминалось, при использовании индексирования следует позаботиться о том, чтобы индексы оставались в диапазоне допустимых значений (см. раздел 3.2.3, стр. 139). В этой программе проверка допустимости подразумевает принадлежность оценки к диапазону 0–100. Таким образом, можно использовать индексы от 0 до 10. Они расположены в пределах от 0 до `scores.size() - 1`.

Индексация не добавляет элементов

Новички в C++ иногда полагают, что индексирование вектора позволяет добавлять в него элементы, но это не так. Следующий код намеревается добавить десять элементов в вектор `ivec`:

```
vector<int> ivec; // пустой вектор
for (decltype(ivec.size()) ix = 0; ix != 10; ++ix)
    ivec[ix] = ix; // катастрофа: ivec не имеет элементов
```

Причина ошибки — вектор `ivec` пуст; в нем нет никаких элементов для индексирования! Как уже упоминалось, правильный цикл использовал бы функцию `push_back()`:

```
for (decltype(ivec.size()) ix = 0; ix != 10; ++ix)
    ivec.push_back(ix); // ok: добавляет новый элемент со значением ix
```



ВНИМАНИЕ

Оператор индексирования вектора (и строки) лишь выбирает существующий элемент; он не может добавить новый элемент.

ВНИМАНИЕ! ИНДЕКСИРОВАТЬ МОЖНО ЛИШЬ СУЩЕСТВУЮЩИЕ ЭЛЕМЕНТЫ!

Очень важно понять, что оператор индексирования (`[]`) можно использовать для доступа только к фактически существующим элементам. Рассмотрим пример.

```
vector<int> ivec;           // пустой вектор
cout << ivec[0];           // ошибка: ivec не имеет элементов!

vector<int> ivec2(10);     // вектор из 10 элементов
cout << ivec2[10];        // ошибка: ivec2 имеет элементы 0...9
```

Попытка обращения к несуществующему элементу является серьезной ошибкой, которую вряд ли обнаружит компилятор. В результате будет получено случайное значение.

Попытка индексирования несуществующих элементов, к сожалению, является весьма распространенной и грубой ошибкой программирования. Так называемая ошибка *переполнения буфера* (buffer overflow) — результат индексирования несуществующих элементов. Такие ошибки являются наиболее распространенной причиной проблем защиты приложений.



Наилучший способ гарантировать невыход индекса из диапазона — это избежать индексации вообще. Для этого везде, где только возможно, следует использовать серийный оператор `for`.

Упражнения раздела 3.3.3

Упражнение 3.16. Напишите программу, выводящую размер и содержимое вектора из упражнения 3.13. Проверьте правильность своих ответов на это упражнение. При неправильных ответах повторно изучите раздел 3.3.1 (стр. 142).

Упражнение 3.17. Прочитайте последовательность слов из потока `cin` и сохраните их в векторе. Прочитав все слова, обработайте вектор и переведите символы каждого слова в верхний регистр. Отобразите преобразованные элементы по восемь слов на строку.

Упражнение 3.18. Корректна ли следующая программа? Если нет, то как ее исправить?

```
vector<int> ivec;
ivec[0] = 42;
```

Упражнение 3.19. Укажите три способа определения вектора и заполнения его десятью элементами со значением 42. Укажите, есть ли предпочтительный способ для этого и почему.

Упражнение 3.20. Прочитайте набор целых чисел в вектор. Отобразите сумму каждой пары соседних элементов. Измените программу так, чтобы она отображала сумму первого и последнего элементов, затем сумму второго и предпоследнего и т.д.



3.4. Знакомство с итераторами

Хотя для доступа к символам строки или элементам вектора можно использовать индексирование, для этого существует и более общий механизм — *итераторы* (iterator). Как будет продемонстрировано в части II, кроме векторов библиотека предоставляет несколько других видов контейнеров. У всех библиотечных контейнеров есть итераторы, но только некоторые из них поддерживают оператор индексирования. С технической точки зрения тип `string` не является контейнерным, но он поддерживает большинство контейнерных операций. Как уже упоминалось, и строки, и векторы предоставляют оператор индексирования. У них также есть итераторы.

Как и указатели (см. раздел 2.3.2, стр. 87), итераторы обеспечивают косвенный доступ к объекту. В случае итератора этим объектом является элемент в контейнере или символ в строке. Итератор позволяет выбрать элемент, а также поддерживает операции перемещения с одного элемента на другой. Подобно указателям, итератор может быть допустим или недопустим. Допустимый итератор указывает либо на элемент, либо на позицию за последним элементом в контейнере. Все другие значения итератора недопустимы.



3.4.1. Использование итераторов

В отличие от указателей, для получения итератора не нужно использовать оператор обращения к адресу. Для этого обладающие итераторами типы имеют члены, возвращающие эти итераторы. В частности, они обладают функциями-членами `begin()` и `end()`. Функция-член `begin()` возвращает итератор, который обозначает первый элемент (или первый символ), если он есть.

```
// типы b и e определяют компилятор; см. раздел 2.5.2, стр. 107
// b обозначает первый элемент rjyntqythf v, а e - элемент
// после последнего
auto b = v.begin(), e = v.end(); // b и e имеют одинаковый тип
```

Итератор, возвращенный функцией `end()`, указывает на следующую позицию за концом контейнера (или строки). Этот итератор обозначает несуществующий элемент за концом контейнера. Он используется как индикатор, означающий, что обработаны все элементы. Итератор, возвращенный функцией `end()`, называют *итератором после конца* (off-the-end iterator), или сокращенно *итератором end*. Если контейнер пуст, функция `begin()` возвращает тот же итератор, что и функция `end()`.



Если контейнер пуст, возвращаемые функциями `begin()` и `end()` итераторы совпадают и, оба являются итератором после конца.

Обычно точный тип, который имеет итератор, неизвестен (да и не нужен). В этом примере при определении итераторов `b` и `e` использовался спецификатор `auto` (см. раздел 2.5.2, стр. 107). В результате тип этих переменных будет совпадать с возвращаемыми функциями-членами `begin()` и `end()` соответственно. Не будем пока распространяться об этих типах до стр. 156.

Операции с итераторами

Итераторы поддерживают лишь несколько операций, которые перечислены в табл. 3.6. Два допустимых итератора можно сравнить при помощи операторов `==` и `!=`. Итераторы равны, если они указывают на тот же элемент или если оба они указывают на позицию после конца того же контейнера. В противном случае они не равны.

Таблица 3.6. Стандартные операции с итераторами контейнера

<code>*iter</code>	Возвращает ссылку на элемент, обозначенный итератором <code>iter</code>
<code>iter->mem</code>	Обращение к значению итератора <code>iter</code> и выборка члена <code>mem</code> основного элемента. Эквивалент <code>(*iter).mem</code>
<code>++iter</code>	Инкремент итератора <code>iter</code> для обращения к следующему элементу контейнера
<code>--iter</code>	Декремент итератора <code>iter</code> для обращения к предыдущему элементу контейнера
<code>iter1 == iter2</code> <code>iter1 != iter2</code>	Сравнивает два итератора на равенство (неравенство). Два итератора равны, если они указывают на тот же элемент или на следующий элемент после конца того же контейнера

Подобно указателям, к значению итератора можно обратиться, чтобы получить элемент, на который он ссылается. Кроме того, подобно указателям, можно обратиться к значению только допустимого итератора, который обозначает некий элемент (см. раздел 2.3.2, стр. 89). Результат обращения к значению недопустимого итератора или итератора после конца непредсказуем.

Перепишем программу из раздела 3.2.3 (стр. 138), преобразующую строчные символы строки в прописные, с использованием итератора вместо индексирования:

```
string s("some string");
if (s.begin() != s.end()) { // удостовериться, что строка s не пуста
    auto it = s.begin();    // it указывает на первый символ строки s
    *it = toupper(*it);     // текущий символ в верхний регистр
}
```

Как и в первоначальной программе, сначала удостоверимся, что строка `s` не пуста. В данном случае для этого сравниваются итераторы, возвращенные функциями `begin()` и `end()`. Эти итераторы равны, если строка пуста. Если они не равны, то в строке `s` есть по крайней мере один символ.

В теле оператора `if` функция `begin()` возвращает итератор на первый символ, который присваивается переменной `it`. Обращение к значению этого итератора и передача его функции `toupper()` позволяет перевести данный символ в верхний регистр. Кроме того, обращение к значению итератора `it` слева от оператора присвоения позволяет присвоить символ, возвращенный функцией `toupper()`, первому символу строки `s`. Как и в первоначальной программе, вывод будет таким:

```
Some string
```

Перемещение итератора с одного элемента на другой

Итераторы используют оператор инкремента (оператор `++`) (см. раздел 1.4.1, стр. 38) для перемещения с одного элемента на следующий. Операция приращения итератора логически подобна приращению целого числа. В случае целых чисел результатом будет целочисленное значение на единицу больше 1. В случае итераторов результатом будет перемещение итератора на одну позицию.



Поскольку итератор, возвращенный функцией `end()`, не указывает на элемент, он не допускает ни приращения, ни обращения к значению.

Перепишем программу, изменяющую регистр первого слова в строке, с использованием итератора.

```
// обрабатывать символы, пока они не исчерпаются, или не встретится пробел
for (auto it = s.begin(); it != s.end() && !isspace(*it); ++it)
    *it = toupper(*it); // преобразовать в верхний регистр
```

Этот цикл, подобно таковому в разделе 3.2.3 (стр. 138), перебирает символы строки `s`, останавливаясь, когда встречается пробел. Но данный цикл использует для этого итератор, а не индексирование.

Цикл начинается с инициализации итератора `it` результатом вызова функции `s.begin()`, чтобы он указывал на первый символ строки `s` (если он есть). Условие проверяет, не достиг ли итератор `it` конца строки (`s.end()`). Если это не так, то проверяется следующее условие, где обращение к значению итератора `it`, возвращающее текущий символ, передается функции `isspace()`, чтобы выяснить, не пробел ли это. В конце каждой итерации выполняется оператор `++it`, чтобы переместить итератор на следующий символ строки `s`.

У этого цикла то же тело, что и у последнего оператора `if` предыдущей программы. Обращение к значению итератора `it` используется и для переда-

чи текущего символа функции `toupper()`, и для присвоения полученного результата символу, на который указывает итератор `it`.

Ключевая концепция. Обобщенное программирование

Программисты, перешедшие на язык C++ с языка C или Java, могли бы быть удивлены тем, что в данном цикле `for` был использован оператор `!=`, а не `<`. Программисты C++ используют оператор `!=` исключительно по привычке. По этой же причине они используют итераторы, а не индексирование: этот стиль программирования одинаково хорошо применим к контейнерам различных видов, предоставляемых библиотекой.

Как уже упоминалось, только у некоторых библиотечных типов, `vector` и `string`, есть оператор индексирования. Тем не менее у всех библиотечных контейнеров есть итераторы, для которых определены операторы `==` и `!=`. Однако большинство их итераторов не имеют оператора `<`. При обычном использовании итераторов и оператора `!=` можно не заботиться о точном типе обрабатываемого контейнера.

Типы итераторов

Подобно тому, как не всегда известен точный тип `size_type` элемента вектора или строки (см. раздел 3.2.2, стр. 131), мы обычно не знаем (да и не обязаны знать) точный тип итератора. Как и в случае с типом `size_type`, библиотечные типы, у которых есть итераторы, определяют типы по имени `iterator` и `const_iterator`, которые представляют фактические типы итераторов.

```
vector<int>::iterator it;           // it позволяет читать и записывать
                                   // в элементы вектора vector<int>
string::iterator it2;              // it2 позволяет читать и записывать
                                   // символы в строку
vector<int>::const_iterator it3;    // it3 позволяет читать, но не
                                   // записывать элементы
string::const_iterator it4;        // it4 позволяет читать, но не
                                   // записывать символы
```

Тип `const_iterator` ведет себя как константный указатель (см. раздел 2.4.2 стр. 99). Как и константный указатель, тип `const_iterator` позволяет читать, но не писать в элемент, на который он указывает; объект типа `iterator` позволяет и читать, и записывать. Если вектор или строка являются константой, можно использовать итератор только типа `const_iterator`. Если вектор или строка не являются константой, можно использовать итератор и типа `iterator`, и типа `const_iterator`.

ТЕРМИНОЛОГИЯ. ИТЕРАТОРЫ И ТИПЫ ИТЕРАТОРОВ

Термин *итератор* (iterator) используется для трех разных сущностей. Речь могла бы идти о *концепции* итератора, или о *типе* iterator, определенном классом контейнера, или об *объекте* итератора.

Следует уяснить, что существует целый набор типов, связанных концептуально. Тип относится к итераторам, если он поддерживает общепринятый набор функций. Эти функции позволяют обращаться к элементу в контейнере и переходить с одного элемента на другой.

Каждый класс контейнера определяет тип по имени iterator; который обеспечивает действия концептуального итератора.

Функции begin() и end()

Тип, возвращаемый функциями begin() и end(), зависит от константности объекта, для которого они были вызваны. Если объект является константой, то функции begin() и end() возвращают итератор типа const_iterator; если объект не константа, они возвращают итератор типа iterator.

```
vector<int> v;
const vector<int> cv;
auto it1 = v.begin(); // it1 имеет тип vector<int>::iterator
auto it2 = cv.begin(); // it2 имеет тип vector<int>::const_iterator
```

C++ 11 Зачастую это стандартное поведение желательно изменить. По причинам, рассматриваемым в разделе 6.2.3 (стр. 281), обычно лучше использовать константный тип (такой как const_iterator), когда необходимо только читать, но не записывать в объект. Чтобы позволить специально задать тип const_iterator, новый стандарт вводит две новые функции, cbegin() и cend():

```
auto it3 = v.cbegin(); // it3 имеет тип vector<int>::const_iterator
```

Подобно функциям-членам begin() и end(), эти функции-члены возвращают итераторы на первый и следующий после последнего элементы контейнера. Но независимо от того, является ли вектор (или строка) константой, они возвращают итератор типа const_iterator.

Объединение обращения к значению и доступа к члену

При обращении к значению итератора получается объект, на который указывает итератор. Если этот объект имеет тип класса, то может понадобиться доступ к члену полученного объекта. Например, если есть вектор строк, то может понадобиться узнать, не пуст ли некий элемент. С учетом, что it — это итератор данного вектора, можно следующим образом проверить, не пуста ли строка, на которую он указывает:

```
(*it).empty()
```

По причинам, рассматриваемым в разделе 4.1.2 (стр. 190), круглые скобки в части `(*it).empty()` необходимы. Круглые скобки требуют применить оператор обращения к значению к итератору `it`, а к результату применить точечный оператор (см. раздел 1.5.2, стр. 51). Без круглых скобок точечный оператор относился бы к итератору `it`, а не к полученному объекту.

```
(*it).empty() // обращение к значению it и вызов функции-члена empty()
               // полученного объекта
*it.empty()   // ошибка: попытка вызова функции-члена empty() итератора it
               // но итератор it не имеет функции-члена empty()
```

Второе выражение интерпретируется как запрос на выполнение функции-члена `empty()` объекта `it`. Но `it` — это итератор, и он не имеет такой функции. Следовательно, второе выражение ошибочно.

Чтобы упростить такие выражения, язык предоставляет *оператор стрелки* (`arrow operator`) (оператор `->`). Оператор стрелки объединяет обращение к значению и доступ к члену. Таким образом, выражение `it->mem` является синонимом выражения `(*it).mem`.

Предположим, например, что имеется вектор `vector<string>` по имени `text`, содержащий данные из текстового файла. Каждый элемент вектора — это либо предложение, либо пустая строка, представляющая конец абзаца. Если необходимо отобразить содержимое первого параграфа из вектора `text`, то можно было бы написать цикл, который перебирает вектор `text`, пока не встретится пустой элемент.

```
// отобразить каждую строку вектора text до первой пустой строки
for (auto it = text.cbegin();
     it != text.cend() && !it->empty(); ++it)
    cout << *it << endl;
```

Код начинается с инициализации итератора `it` указанием на первый элемент вектора `text`. Цикл продолжается до тех пор, пока не будут обработаны все элементы вектора `text` или пока не встретится пустой элемент. Пока есть элементы и текущий элемент не пуст, он отображается. Следует заметить, что, поскольку цикл только читает элементы, но не записывает их, здесь для управления итерацией используются функции `cbegin()` и `cend()`.

Некоторые операции с векторами делают итераторы недопустимыми

В разделе 3.3.2 (стр. 148) упоминался тот факт, что векторы способны расти динамически. Обращалось также внимание на то, что нельзя добавлять элементы в вектор в цикле серийного оператора `for`. Еще одно замечание: любая операция, такая как вызов функции `push_back()`, изменяет размер вектора и способна сделать недопустимыми все итераторы данного вектора. Более подробная информация по этой теме приведена в разделе 9.3.6 (стр. 453).



На настоящий момент достаточно знать, что использующие итераторы цикла не должны добавлять элементы в контейнер, с которым связаны итераторы.



3.4.2. Арифметические действия с итераторами

Инкремент итератора перемещает его на один элемент. Инкремент поддерживают итераторы всех библиотечных контейнеров. Аналогично операторы `==` и `!=` можно использовать для сравнения двух допустимых итераторов (см. раздел 3.4, стр. 153) любых библиотечных контейнеров.

Итераторы строк и векторов поддерживают дополнительные операции, позволяющие перемещать итераторы на несколько позиций за раз. Они также поддерживают все операторы сравнения. Эти операторы зачастую называют *арифметическими действиями с итераторами* (*iterator arithmetic*). Они приведены в табл. 3.7.

Таблица 3.7. Операции с итераторами векторов и строк

<code>iter + n</code>	Добавление (вычитание) целочисленного значения <code>n</code> к (из) итератору возвращает итератор, указывающий на элемент <code>n</code> позиций вперед (назад) в пределах контейнера. Полученный итератор должен указывать на элемент или на следующую позицию за концом того же контейнера
<code>iter - n</code>	
<code>iter1 += n</code>	Составные операторы присвоения со сложением и вычитанием итератора. Присваивает итератору <code>iter1</code> значение на <code>n</code> позиций больше или меньше предыдущего
<code>iter1 -= n</code>	
<code>iter1 - iter2</code>	Вычитание двух итераторов возвращает значение, которое, будучи добавлено к правому итератору, вернет левый. Итераторы должны указывать на элементы или на следующую позицию за концом того же контейнера
<code>>, >=, <, <=</code>	Операторы сравнения итераторов. Один итератор меньше другого, если он указывает на элемент, расположенный в контейнере ближе к началу. Итераторы должны указывать на элементы или на следующую позицию за концом того же контейнера

Арифметические операции с итераторами

К итератору можно добавить (или вычесть из него) целочисленное значение. Это вернет итератор, перемещенный на соответствующее количество позиций вперед (или назад). При добавлении или вычитании целочисленного значения из итератора результат должен указывать на элемент в том же векторе (или строке) или на следующую позицию за концом того же вектора (или строки). В качестве примера вычислим итератор на элемент, ближайший к середине вектора:

```
// вычислить итератор на элемент, ближайший к середине вектора vi
auto mid = vi.begin() + vi.size() / 2;
```

Если у вектора `vi` 20 элементов, то результатом `vi.size()/2` будет 10. В данном случае переменной `mid` будет присвоено значение, равное `vi.begin() + 10`. С учетом, что нумерация индексов начинается с 0, это тот же элемент, что и `vi[10]`, т.е. элемент на десять позиций от начала.

Кроме сравнения двух итераторов на равенство, итераторы векторов и строк можно сравнить при помощи операторов сравнения (`<`, `<=`, `>`, `>=`). Итераторы должны быть допустимы, т.е. должны обозначать элементы (или следующую позицию за концом) того же вектора или строки. Предположим, например, что `it` является итератором в том же векторе, что и `mid`. Следующим образом можно проверить, указывает ли итератор `it` на элемент до или после итератора `mid`:

```
if (it < mid)
// обработать элементы в первой половине вектора vi
```

Можно также вычесть два итератора, если они указывают на элементы (или следующую позицию за концом) того же вектора или строки. Результат — дистанция между итераторами. Под дистанцией подразумевается значение, на которое следует изменить один итератор, чтобы получить другой. Результат имеет целочисленный знаковый тип `difference_type`. Тип `difference_type` определен и для вектора, и для строки. Этот тип знаковый, поскольку результатом вычитания может оказаться отрицательное значение.

Использование арифметических действий с итераторами

Классическим алгоритмом, использующим арифметические действия с итераторами, является *двоичный поиск* (binary search). Двоичный (бинарный) поиск ищет специфическое значение в отсортированной последовательности. Алгоритм работает так: сначала исследуется элемент, ближайший к середине последовательности. Если это искомый элемент, работа закончена. В противном случае, если этот элемент меньше искомого, поиск продолжается только среди элементов после исследованного. Если средний элемент больше искомого, поиск продолжается только в первой половине. Вычисляется новый средний элемент оставшегося диапазона, и действия продолжают, пока искомый элемент не будет найден или пока не исчерпаются элементы.

Используя итераторы, двоичный поиск можно реализовать следующим образом:

```
// текст должен быть отсортирован
// beg и end ограничивают диапазон, в котором осуществляется поиск
auto beg = text.begin(), end = text.end();
auto mid = text.begin() + (end - beg)/2; // исходная середина
// пока еще есть элементы и искомый не найден
while (mid != end && *mid != sought) {
    if (sought < *mid) // находится ли искомый элемент в первой половине?
        end = mid;    // если да, то изменить диапазон, игнорируя вторую
```

```

// половину
else // искомый элемент во второй половине
    beg = mid + 1; // начать поиск с элемента сразу после середины
    mid = beg + (end - beg) / 2; // новая середина
}

```

Код начинается с определения трех итераторов: `beg` будет первым элементом в диапазоне, `end` — элементом после последнего, а `mid` — ближайшим к середине. Инициализируем эти итераторы значениями, охватывающими весь диапазон вектора `vector<string>` по имени `text`.

Сначала цикл проверяет, не пуст ли диапазон. Если значение итератора `mid` равно текущему значению итератора `end`, то элементы для поиска исчерпаны. В таком случае условие ложно и цикл `while` завершается. В противном случае итератор `mid` указывает на элемент, который проверяется на соответствие искомому. Если это так, то цикл завершается.

Если элементы все еще есть, код в цикле `while` корректирует диапазон, перемещая итератор `end` или `beg`. Если обозначенный итератором `mid` элемент больше, чем `sought`, то если искомый элемент и есть в векторе, он находится перед элементом, обозначенным итератором `mid`. Поэтому можно игнорировать элементы после середины, что мы и делаем, присваивая значение итератора `mid` итератору `end`. Если значение `*mid` меньше, чем `sought`, элемент должен быть в диапазоне элементов после обозначенного итератором `mid`. В данном случае диапазон корректируется присвоением итератору `beg` позиции сразу после той, на которую указывает итератор `mid`. Уже известно, что `mid` не указывает на искомый элемент, поэтому его можно исключить из диапазона.

В конце цикла `while` итератор `mid` будет равен итератору `end` либо будет указывать на искомый элемент. Если итератор `mid` равен `end`, то искомого элемента нет в векторе `text`.

Упражнения раздела 3.4.2

Упражнение 3.24. Переделайте последнее упражнение раздела 3.3.3 (стр. 152) с использованием итераторов.

Упражнение 3.25. Перепишите программу кластеризации оценок из раздела 3.3.3 (стр. 150) с использованием итераторов вместо индексации.

Упражнение 3.26. Почему в программе двоичного поиска на стр. 160 использован код `mid = beg + (end - beg) / 2;`, а не `mid = (beg + end) / 2;`?

3.5. Массивы

Массив (`array`) — это структура данных, подобная библиотечному типу `vector` (см. раздел 3.3, стр. 141), но с другим соотношением между производи-

тельностью и гибкостью. Как и вектор, массив является контейнером безымянных объектов одинакового типа, к которым обращаются по позиции. В отличие от вектора, массивы имеют фиксированный размер; добавлять элементы к массиву нельзя. Поскольку размеры массивов постоянны, они иногда обеспечивают лучшую производительность во время выполнения приложений. Но это преимущество приобретается за счет потери гибкости.



Если вы не знаете точно, сколько элементов необходимо, используйте вектор.

3.5.1. Определение и инициализация встроенных массивов

Массив является составным типом (см. раздел 2.3 стр. 84). Оператор объявления массива имеет форму `a[d]`, где `a` — имя; `d` — размерность определяемого массива. Размерность задает количество элементов массива, она должна быть больше нуля. Количество элементов — это часть типа массива, поэтому она должна быть известна на момент компиляции. Следовательно, размерность должна быть константным выражением (см. раздел 2.4.4, стр. 103).

```
unsigned cnt = 42;           // неконстантное выражение
constexpr unsigned sz = 42; // константное выражение
                             // constexpr см. р. 2.4.4 стр. 103
int arr[10];                // массив десяти целых чисел
int *parr[sz];              // массив 42 указателей на int
string bad[cnt];            // ошибка: cnt неконстантное выражение
string strs[get_size()];    // ok, если get_size - constexpr,
                             // в противном случае - ошибка
```

По умолчанию элементы массива инициализируются значением по умолчанию (раздел 2.2.1, стр. 77).



Подобно переменным встроенного типа, инициализированный по умолчанию массив встроенного типа, определенный в функции, будет содержать неопределенные значения.

При определении массива необходимо указать тип его элементов. Нельзя использовать спецификатор `auto` для вывода типа из списка инициализаторов. Подобно вектору, массив содержит объекты. Таким образом, невозможен массив ссылок.

Явная инициализация элементов массива

Массив допускает списочную инициализацию (см. раздел 3.3.1, стр. 143) элементов. В этом случае размерность можно опустить. Если размерность отсутствует, компилятор выводит ее из количества инициализаторов. Если размерность определена, количество инициализаторов не должно превышать ее.

Если размерность больше количества инициализаторов, то инициализаторы используются для первых элементов, а остальные инициализируются по умолчанию (см. раздел 3.3.1, стр. 143):

```
const unsigned sz = 3;
int ial[sz] = {0,1,2};           // массив из трех целых чисел со
                                // значениями 0, 1, 2
int a2[] = {0, 1, 2};           // массив размером 3 элемента
int a3[5] = {0, 1, 2};          // эквивалент a3[] = {0, 1, 2, 0, 0}
string a4[3] = {"hi", "bye"};   // эквивалент a4[] = {"hi", "bye", ""}
int a5[2] = {0,1,2};            // ошибка: слишком много инициализаторов
```

Особенности символьных массивов

У символьных массивов есть дополнительная форма инициализации: строковым литералом (см. раздел 2.1.3 стр. 70). Используя эту форму инициализации, следует помнить, что строковые литералы заканчиваются нулевым символом. Этот нулевой символ копируется в массив наряду с символами литерала.

```
char a1[] = {'C', '+', '+'};    // списочная инициализация без
                                // нулевого символа
char a2[] = {'C', '+', '+', '\0'}; // списочная инициализация с явным
                                // нулевым символом
char a3[] = "C++";              // нулевой символ добавляется
                                // автоматически
const char a4[6] = "Daniel";    // ошибка: нет места для нулевого
                                // символа!
```

Массив `a1` имеет размерность 3; массивы `a2` и `a3` — размерности 4. Определение массива `a4` ошибочно. Хотя литерал содержит только шесть явных символов, массив `a4` должен иметь по крайней мере семь элементов, т.е. шесть для самого литерала и один для нулевого символа.

Не допускается ни копирование, ни присвоение

Нельзя инициализировать массив как копию другого массива, не допустимо также присвоение одного массива другому.

```
int a[] = {0, 1, 2}; // массив из трех целых чисел
int a2[] = a;        // ошибка: нельзя инициализировать один массив
                    // другим
a2 = a;              // ошибка: нельзя присваивать один массив другому
```



ВНИМАНИЕ

Некоторые компиляторы допускают присвоение массивов при применении *расширения компилятора* (compiler extension). Как правило, использования нестандартных средств следует избегать, поскольку они не будут работать на других компиляторах.

Понятие сложных объявлений массива

Как и векторы, массивы способны содержать объекты большинства типов. Например, может быть массив указателей. Поскольку массив — это объект,

можно определять и указатели, и ссылки на массивы. Определение массива, содержащего указатели, довольно просто, определение указателя или ссылки на массив немного сложнее.

```
int *ptrs[10];           // ptrs массив десяти указателей на int
int &refs[10] = /* ? */; // ошибка: массив ссылок невозможен
int (*Parray)[10] = &arr; // Parray указывает на массив из десяти int
int (&arrRef)[10] = arr;  // arrRef ссылается на массив из десяти ints
```

Обычно модификаторы типа читают справа налево. Читаем определение `ptrs` справа налево (см. раздел 2.3.3, стр. 95): определить массив размером 10 по имени `ptrs` для хранения указателей на тип `int`.

Определение `Parray` также стоит читать справа налево. Поскольку размерность массива следует за объявляемым именем, объявление массива может быть легче читать изнутри наружу, а не справа налево. Так намного проще понять тип `Parray`. Объявление начинается с круглых скобок вокруг части `*Parray`, означающей, что `Parray` — указатель. Глядя направо, можно заметить, что указатель `Parray` указывает на массив размером 10. Глядя влево, можно заметить, что элементами этого массива являются целые числа. Таким образом, `Parray` — это указатель на массив из десяти целых чисел. Точно так же часть `(&arrRef)` означает, что `arrRef` — это ссылка, а типом, на который она ссылается, является массив размером 10, хранящий элементы типа `int`.

Конечно, нет никаких ограничений на количество применяемых модификаторов типа.

```
int *(&array)[10] = ptrs; // array - ссылка на массив из десяти указателей
```

Читая это объявление изнутри наружу, можно заметить, что `array` — это ссылка. Глядя направо, можно заметить, что объект, на который ссылается `array`, является массивом размером 10. Глядя влево, можно заметить, что типом элемента является указатель на тип `int`. Таким образом, `array` — это ссылка на массив десяти указателей.



Зачастую объявление массива может быть проще понять, начав его чтение с имени массива и продолжив его изнутри наружу.

Упражнения раздела 3.5.1

Упражнение 3.27. Предположим, что функция `txt_size()` на получает никаких аргументов и возвращают значение типа `int`. Объясните, какие из следующих определений недопустимы и почему?

```
unsigned buf_size = 1024;
(a) int ia[buf_size];      (b) int ia[4 * 7 - 14];
(c) int ia[txt_size()];    (d) char st[11] = "fundamental";
```

Упражнение 3.28. Какие значения содержатся в следующих массивах?

```
string sa[10];
int ia[10];
int main() {
    string sa2[10];
    int    ia2[10];
}
```

Упражнение 3.29. Перечислите некоторые из недостатков использования массива вместо вектора.

3.5.2. Доступ к элементам массива

Подобно библиотечным типам `vector` и `string`, для доступа к элементам массива можно использовать серийный оператор `for` или *оператор индексирования* (`[]`) (subscript). Как обычно, индексы начинаются с 0. Для массива из десяти элементов используются индексы от 0 до 9, а не от 1 до 10.

При использовании переменной для индексирования массива ее обычно определяют как имеющую тип `size_t`. Тип `size_t` — это машинозависимый беззнаковый тип, гарантированно достаточно большой для содержания размера любого объекта в памяти. Тип `size_t` определен в заголовке `cstdint`, который является версией C++ заголовка `stdint.h` библиотеки C.

За исключением фиксированного размера, массивы используются подобно векторам. Например, можно повторно реализовать программу оценок из раздела 3.3.3 (стр. 150), используя для хранения счетчиков кластеров массив.

```
// подсчет количества оценок в кластере по десять: 0--9,
// 10--19, ... 90--99, 100
unsigned scores[11] = {}; // 11 ячеек, все со значением 0
unsigned grade;
while (cin >> grade) {
    if (grade <= 100)
        ++scores[grade/10]; // приращение счетчика текущего кластера
}
```

Единственное очевидное различие между этой программой и приведенной на стр. 150 в объявлении массива `scores`. В данной программе это массив из 11 элементов типа `unsigned`. Не столь очевидно то различие, что оператор индексирования в данной программе тот, который определен как часть языка. Этот оператор применяется с операндами типа массива. Оператор индексирования, используемый в программе на стр. 150, был определен библиотечным шаблоном `vector` и применялся к операндам типа `vector`.

Как и в случае строк или векторов, для перебора всего массива лучше использовать серийный оператор `for`. Например, все содержимое массива `scores` можно отобразить следующим образом:

```
for (auto i : scores) // для каждого счетчика в scores
    cout << i << " "; // отобразить его значение
cout << endl;
```

Поскольку размерность является частью типа каждого массива, системе известно количество элементов в массиве `scores`. Используя средства серийного оператора `for`, перебором можно управлять и не самостоятельно.

Проверка значений индекса

Как и в случае со строкой и вектором, ответственность за невыход индекса за пределы массива лежит на самом программисте. Он сам должен гарантировать, что значение индекса будет больше или равно нулю, но не больше размера массива. Ничто не мешает программе перешагнуть границу массива, кроме осторожности и внимания разработчика, а также полной проверки кода. В противном случае программа будет компилироваться и выполняться правильно, но все же содержать скрытую ошибку, способную проявиться в наименее подходящий момент.



Наиболее распространенным источником проблем защиты приложений является ошибка переполнения буфера. Причиной такой ошибки является отсутствие в программе проверки индекса, в результате чего программа ошибочно использует память вне диапазона массива или подобной структуры данных.

Упражнения раздела 3.5.2

Упражнение 3.30. Выявите ошибки индексации в следующем коде:

```
constexpr size_t array_size = 10;
int ia[array_size];
for (size_t ix = 1; ix <= array_size; ++ix)
    ia[ix] = ix;
```

Упражнение 3.31. Напишите программу, где определен массив из десяти целых чисел, каждому элементу которого присвоено значение, соответствующее его позиции в массиве.

Упражнение 3.32. Скопируйте массив, определенный в предыдущем упражнении, в другой массив. Перезапишите эту программу так, чтобы использовались векторы.

Упражнение 3.33. Что будет, если не инициализировать массив `scores` в программе на стр. 165?

3.5.3. Указатели и массивы

Указатели и массивы в языке C++ тесно связаны. В частности, как будет продемонстрировано вскоре, при использовании массивов компилятор обычно преобразует их в указатель.

Обычно указатель на объект получают при помощи оператора обращения к адресу (см. раздел 2.3.2, стр. 87). По правде говоря, оператор обращения к адресу может быть применен к любому объекту, а элементы в массиве — объекты. При индексировании массива результатом является объект в этой области массива. Подобно любым другим объектам, указатель на элемент массива можно получить из адреса этого элемента:

```
string nums[] = {"one", "two", "three"}; // массив строк
string *p = &nums[0]; // p указывает на первый элемент массива nums
```

Однако у массивов есть одна особенность — места их использования компилятор автоматически заменяет указателем на первый элемент.

```
string *p2 = nums; // эквивалент p2 = &nums[0]
```



В большинстве выражений, где используется объект типа массива, в действительности используется указатель на первый элемент в этом массиве.

Существует множество свидетельств того факта, что операции с массивами зачастую являются операциями с указателями. Одно из них — при использовании массива как инициализатора переменной, определенной с использованием спецификатора `auto` (см. раздел 2.5.2, стр. 107), выводится тип указателя, а не массива.

```
int ia[] = {0,1,2,3,4,5,6,7,8,9}; // ia - массив из десяти целых чисел
auto ia2(ia); // ia2 - это int*, указывающий на первый элемент в ia
ia2 = 42;      // ошибка: ia2 - указатель нельзя присвоить указателю
               // значение типа int
```

Хотя `ia` является массивом из десяти целых чисел, при его использовании в качестве инициализатора компилятор рассматривает это как следующий код:

```
auto ia2(&ia[0]); // теперь ясно, что ia2 имеет тип int*
```

Следует заметить, что это преобразование не происходит, если используется спецификатор `decltype` (см. раздел 2.5.3, стр. 110). Выражение `decltype(ia)` возвращает массив из десяти целых чисел:

```
// ia3 - массив из десяти целых чисел
decltype(ia) ia3 = {0,1,2,3,4,5,6,7,8,9};
ia3 = p;      // ошибка: невозможно присвоить int* массиву
ia3[4] = i;    // ок: присвоить значение i элементу в массиве ia3
```

Указатели — это итераторы

Указатели, содержащие адреса элементов в массиве, обладают дополнительными возможностями, кроме описанных в разделе 2.3.2 (стр. 86). В частности, указатели на элементы массивов поддерживают те же операции, что и итераторы векторов или строк (см. раздел 3.4, стр. 153). Например, можно использовать оператор инкремента для перемещения с одного элемента массива на следующий:

```
int arr[] = {0,1,2,3,4,5,6,7,8,9};
int *p = arr; // p указывает на первый элемент в arr
++p;          // p указывает на arr[1]
```

Подобно тому, как итераторы можно использовать для перебора элементов вектора, указатели можно использовать для перебора элементов массива. Конечно, для этого нужно получить указатели на первый элемент и элемент, следующий после последнего. Как упоминалось только что, указатель на первый элемент можно получить при помощи самого массива или при обращении к адресу первого элемента. Получить указатель на следующий элемент после последнего можно при помощи другого специального свойства массива. Последний элемент массива `arr` находится в позиции 9, а адрес несуществующего элемента массива, следующего после него, можно получить так:

```
int *e = &arr[10]; // указатель на элемент после последнего в массиве arr
```

Единственное, что можно сделать с этим элементом, так это получить его адрес, чтобы инициализировать указатель `e`. Как и итератор на элемент после конца (см. раздел 3.4.1 стр. 153), указатель на элемент после конца не указывает ни на какой элемент. Поэтому нельзя ни обратиться к его значению, ни прирастить.

Используя эти указатели, можно написать цикл, выводящий элементы массива `arr`.

```
for (int *b = arr; b != e; ++b)
    cout << *b << endl; // вывод элементов arr
```

Библиотечные функции `begin()` и `end()`

C++ 11 Указатель на элемент после конца можно вычислить, но этот подход подвержен ошибкам. Чтобы облегчить и обезопасить использование указателей, новая библиотека предоставляет две функции: `begin()` и `end()`. Эти функции действуют подобно одноименным функциям-членам контейнеров (см. раздел 3.4.1 стр. 153). Однако массивы — не классы, и данные функции не могут быть функциями-членами. Поэтому для работы они получают массив в качестве аргумента.

```
int ia[] = {0,1,2,3,4,5,6,7,8,9}; // ia - массив из десяти целых чисел
int *beg = begin(ia); // указатель на первый элемент массива ia
int *last = end(ia);  // указатель на следующий элемент ia за последним
```

Функция `begin()` возвращает указатель на первый, а функция `end()` на следующий после последнего элемент данного массива. Эти функции определены в заголовке `iterator`.

Используя функции `begin()` и `end()`, довольно просто написать цикл обработки элементов массива. Предположим, например, что массив `arr` содержит значения типа `int`. Первое отрицательное значение в массиве `arr` можно найти следующим образом:

```
// pbegin указывает на первый, а pend на следующий после последнего
// элемент массива arr
int *pbegin = begin(arr), *pend = end(arr);
// найти первый отрицательный элемент, остановиться, если просмотрены
// все элементы
while (pbegin != pend && *pbegin >= 0)
    ++pbegin;
```

Код начинается с определения двух указателей типа `int` по имени `pbegin` и `pend`. Указатель `pbegin` устанавливается на первый элемент массива `arr`, а `pend` — на следующий элемент после последнего. Условие цикла `while` использует указатель `pend`, чтобы узнать, безопасно ли обращаться к значению указателя `pbegin`. Если указатель `pbegin` действительно указывает на элемент, выполняется проверка результата обращения к его значению на наличие отрицательного значения. Если это так, то условие ложно и цикл завершается. В противном случае указатель переводится на следующий элемент.



Указатель на элемент “после последнего” у встроенного массива ведет себя так же, как итератор, возвращенный функцией `end()` вектора. В частности, нельзя ни обратиться к значению такого указателя, ни осуществить его приращение.

Арифметические действия с указателями

Указатели на элементы массива позволяют использовать все операции с итераторами, перечисленные в табл. 3.6 (стр. 154) и 3.7 (стр. 159). Эти операции, обращения к значению, инкремента, сравнения, добавления целочисленного значения, вычитания двух указателей, имеют для указателей на элементы встроенного массива то же значение, что и для итераторов.

Результатом добавления (или вычитания) целочисленного значения к указателю (или из него) является новый указатель, указывающий на элемент, расположенный на заданное количество позиций вперед (или назад) от исходного указателя.

```
constexpr size_t sz = 5;
int arr[sz] = {1, 2, 3, 4, 5};
int *ip = arr;           // эквивалент int *ip = &arr[0]
int *ip2 = ip + 4;       // ip2 указывает на arr[4], последний элемент в arr
```

Результатом добавления 4 к указателю `ip` будет указатель на элемент, расположенный в массиве на четыре позиции далее от того, на который в настоящее время указывает `ip`.

Результатом добавления целочисленного значения к указателю должен быть указатель на элемент (или следующую позицию после конца) в том же массиве:

```
// ok: arr преобразуется в указатель на его первый элемент;
// p указывает на позицию после конца arr
int *p = arr + sz; // использовать осматрительно - не обращаться
                  // к значению!
int *p2 = arr + 10; // ошибка: arr имеет только 5 элементов;
                  // значение p2 неопределенно
```

При сложении `arr` и `sz` компилятор преобразует `arr` в указатель на первый элемент массива `arr`. При добавлении `sz` к этому указателю получается указатель на позицию `sz` (т.е. на позицию 5) этого массива. Таким образом, он указывает на следующую позицию после конца массива `arr`. Вычисление указателя на более чем одну позицию после последнего элемента является ошибкой, хотя компилятор таких ошибок не обнаруживает.

Подобно итераторам, вычитание двух указателей дает дистанцию между ними. Указатели должны указывать на элементы в том же массиве:

```
auto n = end(arr) - begin(arr); // n - 5, количество элементов
                               // массива arr
```

Результат вычитания двух указателей имеет библиотечный тип `ptrdiff_t`. Как и тип `size_t`, тип `ptrdiff_t` является машинозависимым типом, определенным в заголовке `cstdint`. Поскольку вычитание способно вернуть отрицательное значение, тип `ptrdiff_t` — знаковый целочисленный.

Для сравнения указателей на элементы (или позицию за концом) массива можно использовать операторы сравнения. Например, элементы массива `arr` можно перебрать следующим образом:

```
int *b = arr, *e = arr + sz;
while (b < e) {
    // используется *b
    ++b;
}
```

Нельзя использовать операторы сравнения для указателей на два несвязанных объекта.

```
int i = 0, sz = 42;
int *p = &i, *e = &sz;
// неопределенно: p и e не связаны; сравнение бессмысленно!
while (p < e)
```

Хотя на настоящий момент смысл может быть и неясен, но следует заметить, что арифметические действия с указателями допустимы также для нулевых указателей (см. раздел 2.3.2, стр. 89) и для указателей на объекты, не являющиеся массивом. В последнем случае указатели должны указывать на тот же объект или следующий после него. Если `p` — нулевой указатель, то к нему можно добавить (или вычесть) целочисленное константное выражение (см. раздел 2.4.4, стр. 103) со значением 0. Можно также вычесть два нулевых указателя из друг друга, и результатом будет 0.

Взаимодействие обращения к значению с арифметическими действиями с указателями

Результатом добавления целочисленного значения к указателю является указатель. Если полученный указатель указывает на элемент, то к его значению можно обратиться:

```
int ia[] = {0,2,4,6,8}; // массив из 5 элементов типа int
int last = *(ia + 4); // ok: инициализирует last значением ia[4], т.е. 8
```

Выражение `*(ia + 4)` вычисляет адрес четвертого элемента после `ia` и обращается к значению полученного указателя. Это выражение эквивалентно выражению `ia[4]`.

Помните, в разделе 3.4.1 (стр. 158) обращалось внимание на необходимость круглых скобок в выражениях, содержащих оператор обращения к значению и точечный оператор. Аналогично необходимы круглые скобки вокруг части сложения указателей:

```
last = *ia + 4; // ok: last = 4, эквивалент ia[0] + 4
```

Этот код обращается к значению `ia` и добавляет 4 к полученному значению. Причины подобного поведения рассматриваются в разделе 4.1.2 (стр. 190).



Индексирование и указатели

Как уже упоминалось, в большинстве мест, где используется имя массива, в действительности используется указатель на первый элемент этого массива. Одним из мест, где компилятор осуществляет это преобразование, является индексирование массива.

```
int ia[] = {0,2,4,6,8}; // массив из 5 элементов типа int
```

Рассмотрим выражение `ia[0]`, использующее имя массива. При индексировании массива в действительности индексируется указатель на элемент в этом массиве.

```
int i = ia[2]; // ia преобразуется в указатель на первый элемент ia
               // ia[2] выбирает элемент, на который указывает (ia + 2)
```

```
int *p = ia;    // p указывает на первый элемент в массиве ia
i = *(p + 2);  // эквивалент i = ia[2]
```

Оператор индексирования можно использовать для любого указателя, пока он указывает на элемент (или позицию после конца) в массиве.

```
int *p = &ia[2]; // p указывает на элемент с индексом 2
int j = p[1];    // p[1] - эквивалент *(p + 1),
                // p[1] тот же элемент, что и ia[3]
int k = p[-2];   // p[-2] тот же элемент, что и ia[0]
```

Последний пример указывает на важное отличие между массивами и такими библиотечными типами, как `vector` и `string`, у которых есть операторы индексирования. Библиотечные типы требуют, чтобы используемый индекс был беззнаковым значением. Встроенный оператор индексирования этого не требует. Индекс, используемый со встроенным оператором индексирования, может быть отрицательным значением. Конечно, полученный адрес должен указывать на элемент (или позицию после конца) массива, на который указывает первоначальный указатель.



ВНИМАНИЕ

В отличие от индексов для векторов и строк, индекс встроенного массива не является беззнаковым.

Упражнения раздела 3.5.3

Упражнение 3.34. С учетом, что указатели `p1` и `p2` указывают на элементы в том же массиве, что делает следующий код? Какие значения `p1` или `p2` делают этот код недопустимым?

```
p1 += p2 - p1;
```

Упражнение 3.35. Напишите программу, которая использует указатели для обнуления элементов массива.

Упражнение 3.36. Напишите программу, сравнивающую два массива на равенство. Напишите подобную программу для сравнения двух векторов.

3.5.4. Символьные строки в стиле C



ВНИМАНИЕ

Хотя язык C++ поддерживает строки в стиле C, использовать их в программах C++ не следует. Строки в стиле C — на удивление богатый источник разнообразных ошибок и наиболее распространенная причина проблем защиты.

Символьный строковый литерал — это экземпляр более общей конструкции, которую язык C++ унаследовал от языка C: *символьной строки в стиле C* (C-style character string). Строка в стиле C не является типом данных, скорее это соглашение о представлении и использовании символьных строк. Сле-

дующие этому соглашению строки хранятся в символьных массивах и являются строкой с завершающим нулевым символом (null-terminated string). Под завершающим нулевым символом подразумевается, что последний видимый символ в строке сопровождается нулевым символом ('\\0'). Для манипулирования этими строками обычно используются указатели.

Строковые функции библиотеки C

Стандартная библиотека языка C предоставляет набор функций, перечисленных в табл. 3.8, для работы со строками в стиле C. Эти функции определены в заголовке `cstring`, являющемся версией C++ заголовка языка C `string.h`.



Функции из табл. 3.8 не проверяют свои строковые параметры.

Указатель (указатели), передаваемый этим функциям, должен указывать на массив (массивы) с нулевым символом в конце.

```
char ca[] = {'C', '+', '+'}; // без нулевого символа в конце
cout << strlen(ca) << endl; // катастрофа: ca не завершается нулевым
                             // символом
```

В данном случае `ca` — это массив элементов типа `char`, но он не завершается нулевым символом. Результат непредсказуем. Вероятней всего, функция `strlen()` продолжит просматривать память уже за пределами массива `ca`, пока не встретит нулевой символ.

Таблица 3.8. Функции для символьных строк в стиле C

<code>strlen(p)</code>	Возвращает длину строки <code>p</code> без учета нулевого символа
<code>strcmp(p1, p2)</code>	Проверяет равенство строк <code>p1</code> и <code>p2</code> . Возвращает 0, если <code>p1 == p2</code> , положительное значение, если <code>p1 > p2</code> , и отрицательное значение, если <code>p1 < p2</code>
<code>strcat(p1, p2)</code>	Добавляет строку <code>p2</code> к <code>p1</code> . Результат возвращает в строку <code>p1</code>
<code>strcpy(p1, p2)</code>	Копирует строку <code>p2</code> в строку <code>p1</code> . Результат возвращает в строку <code>p1</code>

Сравнение строк

Сравнение двух строк в стиле C осуществляется совсем не так, как сравнение строк библиотечного типа `string`. При сравнении библиотечных строк используются обычные операторы равенства или сравнения:

```
string s1 = "A string example";
string s2 = "A different string";
if (s1 < s2) // ложно: s2 меньше s1
```

Использование этих же операторов для подобным образом определенных строк в стиле C приведет к сравнению значений указателей, а не самих строк.

```
const char ca1[] = "A string example";
const char ca2[] = "A different string";
if (ca1 < ca2) // непредсказуемо: сравниваются два адреса
```

Помните, что при использовании массива в действительности используются указатели на их первый элемент (см. раздел 3.5.3, стр. 167). Следовательно, это условие фактически сравнивает два значения `const char*`. Эти указатели содержат адреса разных объектов, поэтому результат такого сравнения непредсказуем.

Чтобы сравнить строки, а не значения указателей, можем использовать функцию `strcmp()`. Она возвращает значение 0, если строки равны, положительное или отрицательное значение, в зависимости от того, больше ли первая строка второй или меньше.

```
if (strcmp(ca1, ca2) < 0) // то же, что и сравнение строк s1 < s2
```

За размер строки отвечает вызывающая сторона

Конкатенация и копирование строк в стиле C также весьма отличается от таких же операций с библиотечным типом `string`. Например, если необходима конкатенация строк `s1` и `s2`, определенных выше, то это можно сделать так:

```
// инициализировать largeStr результатом конкатенации строки s1,  
// пробела и строки s2  
string largeStr = s1 + " " + s2;
```

Подобное с двумя массивами, `ca1` и `ca2`, было бы ошибкой. Выражение `ca1 + ca2` попытается сложить два указателя, что некорректно и бессмысленно.

Вместо этого можно использовать функции `strcat()` и `strcpy()`. Но чтобы использовать эти функции, им необходимо передать массив для хранения результирующей строки. Передаваемый массив должен быть достаточно большим, чтобы содержать созданную строку, включая нулевой символ в конце. Хотя представленный здесь код следует традиционной схеме, потенциально он может стать причиной серьезной ошибки.

```
// катастрофа, если размер largeStr вычислен ошибочно  
strcpy(largeStr, ca1); // копирует ca1 в largeStr  
strcat(largeStr, " "); // добавляет пробел в конец largeStr  
strcat(largeStr, ca2); // конкатенирует ca2 с largeStr
```

Проблема в том, что можно легко ошибиться в расчете необходимого размера `largeStr`. Кроме того, при каждом изменении значения, которые следует сохранить в `largeStr`, необходимо перепроверить правильность вычисления его размера. К сожалению, код, подобный этому, широко распространен в программах. Такие программы подвержены ошибкам и часто приводят к серьезным проблемам защиты.



Для большинства приложений не только безопасней, но и эффективней использовать библиотечный тип `string`, а не строки в стиле С.

Упражнения раздела 3.5.4

Упражнение 3.37. Что делает следующая программа?

```
const char ca[] = {'h', 'e', 'l', 'l', 'o'};
const char *cp = ca;
while (*cp) {
    cout << *cp << endl;
    ++cp;
}
```

Упражнение 3.38. В этом разделе упоминалось, что не только некорректно, но и бессмысленно пытаться сложить два указателя. Почему сложение двух указателей бессмысленно?

Упражнение 3.39. Напишите программу, сравнивающую две строки. Затем напишите программу, сравнивающую значения двух символьных строк в стиле С.

Упражнение 3.40. Напишите программу, определяющую два символьных массива, инициализированных строковыми литералами. Теперь определите третий символьный массив для содержания результата конкатенации этих двух массивов. Используйте функции `strcpy()` и `strcat()` для копирования этих двух массивов в третий.

3.5.5. Взаимодействие с устаревшим кодом

Множество программ С++ было написано до появления стандартной библиотеки, поэтому они не используют библиотечные типы `string` и `vector`. Кроме того, многие программы С++ взаимодействуют с программами, написанными на языке С или других языках, которые не могут использовать библиотеку С++. Следовательно, программам, написанным на современном языке С++, вероятно, придется взаимодействовать с кодом, который использует символьные строки в стиле С и/или массивы. Библиотека С++ предоставляет средства, облегчающие такое взаимодействие.



Совместное использование библиотечных строк и строк в стиле С

В разделе 3.2.1 (стр. 126) была продемонстрирована возможность инициализации строки класса `string` строковым литералом:

```
string s("Hello World"); // s содержит Hello World
```

В общем, символьный массив с нулевым символом в конце можно использовать везде, где используется строковый литерал.

- Символьный массив с нулевым символом в конце можно использовать для инициализации строки класса `string` или присвоения ей.
- Символьный массив с нулевым символом в конце можно использовать как один из операндов (но не оба) в операторе суммы класса `string` или как первый операнд в составном операторе присвоения (`+=`) класса `string`.

Однако нет никакого простого способа использовать библиотечную строку там, где требуется строка в стиле C. Например, невозможно инициализировать символьный указатель объектом класса `string`. Тем не менее класс `string` обладает функцией-членом `c_str()`, зачастую позволяющей выполнить желаемое.

```
char *str = s; // ошибка: нельзя инициализировать char* из string
const char *str = s.c_str(); // ok
```

Имя функции `c_str()` означает, что она возвращает символьную строку в стиле C. Таким образом, она возвращает указатель на начало символьного массива с нулевым символом в конце, содержащим те же символы, что и строка. Тип указателя `const char*` не позволяет изменять содержимое массива.

Допустимость массива, возвращенного функцией `c_str()`, не гарантируется. Любое последующее использование указателя `s`, способное изменить его значение, может сделать этот массив недопустимым.



ВНИМАНИЕ

Если программа нуждается в продолжительном доступе к содержимому массива, возвращенного функцией `c_str()`, то следует создать его копию.

Использование массива для инициализации вектора

В разделе 3.5.1 (стр. 163) упоминалось о том, что нельзя инициализировать встроенный массив другим массивом. Инициализировать массив из вектора также нельзя. Однако можно использовать массив для инициализации вектора. Для этого необходимо определить адрес первого подлежащего копированию элемента и элемента, следующего за последним.

```
int int_arr[] = {0, 1, 2, 3, 4, 5};
// вектор ivec содержит 6 элементов, каждый из которых является
// копией соответствующего элемента массива int_arr
vector<int> ivec(begin(int_arr), end(int_arr));
```

Два указателя, используемые при создании вектора `ivec`, отмечают диапазон значений, используемых для инициализации его элементов. Второй указатель указывает на следующий элемент после последнего копируемого. В данном случае для передачи указателей на первый и следующий после последне-

го элементы массива `int_arr` использовались библиотечные функции `begin()` и `end()` (см. раздел 3.5.3, стр. 168). В результате вектор `ivec` содержит шесть элементов, значения которых совпадают со значениями соответствующих элементов массива `int_arr`.

Определяемый диапазон может быть также подмножеством массива:

```
// скопировать 3 элемента: int_arr[1], int_arr[2], int_arr[3]
vector<int> subVec(int_arr + 1, int_arr + 4);
```

Этот код создает вектор `subVec` с тремя элементами, значения которых являются копиями значений элементов от `int_arr[1]` до `int_arr[3]`.

СОВЕТ. ИСПОЛЬЗУЙТЕ ВМЕСТО МАССИВОВ БИБЛИОТЕЧНЫЕ ТИПЫ

Указатели и массивы на удивление сильно подвержены ошибкам. Частично проблема в концепции: указатели используются для низкоуровневых манипуляций, в них очень просто сделать тривиальные ошибки. Другие проблемы возникают из-за используемого синтаксиса, особенно синтаксиса объявлений.

Упражнения раздела 3.5.5

Упражнение 3.41. Напишите программу, инициализирующую вектор значениями из массива целых чисел.

Упражнение 3.42. Напишите программу, копирующую вектор целых чисел в массив целых чисел.



3.6. Многомерные массивы

Строго говоря, никаких *многомерных массивов* (multidimensioned array) в языке C++ нет. То, что обычно упоминают как многомерный массив, фактически является массивом массивов. Не забывайте об этом факте, когда будете использовать то, что называют многомерным массивом.

При определении массива, элементы которого являются массивами, указываются две размерности: размерность самого массива и размерность его элементов.

```
int ia[3][4]; // массив из 3 элементов; каждый из которых является
              // массивом из 4 целых чисел
// массив из 10 элементов, каждый из которых является массивом из 20
// элементов, каждый из которых является массивом из 30 целых чисел
int arr[10][20][30] = {0}; // инициализировать все элементы значением 0
```

Как уже упоминалось в разделе 3.5.1 (стр. 162), может быть легче понять эти определения, читая их изнутри наружу. Сначала можно заметить определяемое имя, `ia`, далее видно, что это массив размером 3. Продолжая вправо, ви-

дим, что у элементов массива `ia` также есть размерность. Таким образом, элементы массива `ia` сами являются массивами размером 4. Глядя влево, видно, что типом этих элементов является `int`. Так, `ia` является массивом из трех элементов, каждый из которых является массивом из четырех целых чисел.

Прочитаем определение массива `arr` таким же образом. Сначала увидим, что `arr` — это массив размером 10 элементов. Элементы этого массива сами являются массивами размером 20 элементов. У каждого из этих массивов по 30 элементов типа `int`. Нет предела количеству используемых индексирований. Поэтому вполне может быть массив, элементы которого являются массивами массив, массив, массив и т.д.

В двумерном массиве первую размерность зачастую называют *рядом* (row), а вторую — *столбцом* (column).

Инициализация элементов многомерного массива

Подобно любым массивам, элементы многомерного массива можно инициализировать, предоставив в фигурных скобках список инициализаторов. Многомерные массивы могут быть инициализированы списками значений в фигурных скобках для каждого ряда.

```
int ia[3][4] = {    // три элемента; каждый - массив размером 4
    {0, 1, 2, 3}, // инициализаторы ряда 0
    {4, 5, 6, 7}, // инициализаторы ряда 1
    {8, 9, 10, 11} // инициализаторы ряда 2
};
```

Вложенные фигурные скобки необязательны. Следующая инициализация эквивалентна, хотя и значительно менее очевидна:

```
// эквивалентная инициализация без необязательных вложенных фигурных
// скобок для каждого ряда
int ia[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

Как и в случае одномерных массивов, элементы списка инициализации могут быть пропущены. Следующим образом можно инициализировать только первый элемент каждого ряда:

```
// явная инициализация только нулевого элемента в каждом ряду
int ia[3][4] = {{ 0 }, { 4 }, { 8 }};
```

Остальные элементы инициализируются значением по умолчанию, как и обычные одномерные массивы (см. раздел 3.5.1 стр. 162). Но если опустить вложенные фигурные скобки, то результаты были бы совсем иными:

```
// явная инициализация нулевого ряда; остальные элементы инициализируются
// по умолчанию
int ix[3][4] = {0, 3, 6, 9};
```

Этот код инициализирует элементы первого ряда. Остальные элементы инициализируются значением 0.

Индексация многомерных массивов

Подобно любому другому массиву, для доступа к элементам многомерного массива можно использовать индексирование. При этом для каждой размерности используется отдельный индекс.

Если выражение предоставляет столько же индексов, сколько у массива размерностей, получается элемент с определенным типом. Если предоставить меньше индексов, чем есть размерностей, то результатом будет элемент внутреннего массива по указанному индексу:

```
// присваивает первый элемент массива arr последнему элементу
// в последнем ряду массива ia
ia[2][3] = arr[0][0][0];
int (&row)[4] = ia[1]; // связывает ряд второго массива с четырьмя
                       // элементами массива ia
```

В первом примере предоставляются индексы для всех размерностей обоих массивов. Левая часть, `ia[2]`, возвращает последний ряд массива `ia`. Она возвращает не отдельный элемент массива, а сам массив. Индексируем массив, выбирая элемент `[3]`, являющийся последним элементом данного массива.

Точно так же, правый операнд имеет три размерности. Сначала выбирается массив по индексу `0` из наиболее удаленного массива. Результат этой операции — массив (многомерный) размером `20`. Используя массив размером `30`, извлекаем из этого массива с `20` элементами первый элемент. Затем выбирается первый элемент из полученного массива.

Во втором примере `row` определяется как ссылка на массив из четырех целых чисел. Эта ссылка связывается со вторым рядом массива `ia`.

```
constexpr size_t rowCnt = 3, colCnt = 4;
int ia[rowCnt][colCnt]; // 12 неинициализированных элементов
// для каждого ряда
for (size_t i = 0; i != rowCnt; ++i) {
    // для каждого столбца в ряду
    for (size_t j = 0; j != colCnt; ++j) {
        // присвоить элементу его индекс как значение
        ia[i][j] = i * colCnt + j;
    }
}
```

Внешний цикл `for` перебирает каждый элемент массива `ia`. Внутренний цикл `for` перебирает элементы внутренних массивов. В данном случае каждому элементу присваивается значение его индекса в общем массиве.



Использование серийного оператора `for` с многомерными массивами

По новому стандарту предыдущий цикл можно упростить с помощью серийного оператора `for`:

```

size_t cnt = 0;
for (auto &row : ia)           // для каждого элемента во внешнем массиве
    for (auto &col : row) {    // для каждого элемента во внутреннем массиве
        col = cnt;           // присвоить значение текущему элементу
        ++cnt;               // инкремент cnt
    }

```

Этот цикл присваивает элементам массива `ia` те же значения, что и предыдущий цикл, но на сей раз управление индексами берет на себя система. Значения элементов необходимо изменить, поэтому объявляем управляющие переменные `row` и `col` как ссылки (см. раздел 3.2.3, стр. 138). Первый оператор `for` перебирает элементы массива `ia`, являющиеся массивами из 4 элементов. Таким образом, типом `row` будет ссылка на массив из четырех целых чисел. Второй цикл `for` перебирает каждый из этих массивов по 4 элемента. Следовательно, `col` имеет тип `int&`. На каждой итерации значение `cnt` присваивается следующему элементу массива `ia`, а затем осуществляется инкремент переменной `cnt`.

В предыдущем примере как управляющие переменные цикла использовались ссылки, поскольку элементы массива необходимо было изменять. Однако есть и более серьезная причина для использования ссылок. Рассмотрим в качестве примера следующий цикл:

```

for (const auto &row : ia) // для каждого элемента во внешнем массиве
    for (auto col : row)   // для каждого элемента во внутреннем массиве
        cout << col << endl;

```

Этому циклу запись в элементы не нужна, но все же управляющая переменная внешнего цикла определена как ссылка. Это сделано для того, чтобы избежать преобразования обычного массива в указатель (см. раздел 3.5.3, стр. 167). Если пренебречь ссылкой и написать эти циклы так, то компиляция потерпит неудачу:

```

for (auto row : ia)
    for (auto col : row)

```

Как и прежде, первый цикл `for` перебирает элементы массива `ia`, являющиеся массивами по 4 элемента. Поскольку `row` не ссылка, при его инициализации компилятор преобразует каждый элемент массива (как и любой другой объект типа массива) в указатель на первый элемент этого массива. В результате типом `row` в этом цикле будет `int*`. Внутренний цикл `for` некорректен. Несмотря на намерения разработчика, этот цикл пытается перебрать указатель типа `int*`.



Чтобы использовать многомерный массив в серийном операторе `for`, управляющие переменные всех циклов, кроме самого внутреннего, должны быть ссылками.

Указатели и многомерные массивы

Подобно любым другим массивам, имя многомерного массива автоматически преобразуется в указатель на первый его элемент.



Определяя указатель на многомерный массив, помните, что на самом деле он является массивом массивов.

Поскольку многомерный массив в действительности является массивом массивов, тип указателя, в который преобразуется массив, является типом первого внутреннего массива.

```
int ia[3][4];           // массив размером 3 элемента; каждый элемент - массив
                        // из 4 целых чисел
int (*p)[4] = ia;       // p указывает на массив из четырех целых чисел
p = &ia[2];             // теперь p указывает на последний элемент ia
```

Применяя стратегию из раздела 3.5.1 (стр. 162), начнем рассмотрение с части `(*p)`, гласящей, что `p` — указатель. Глядя вправо, замечаем, что объект, на который указывает указатель `p`, имеет размер 4 элемента, а глядя влево, видим, что типом элемента является `int`. Следовательно, `p` — это указатель на массив из четырех целых чисел.



Круглые скобки в этом объявлении необходимы.

```
int *ip[4];             // массив указателей на int
int (*ip)[4];           // указатель на массив из четырех целых чисел
```



Новый стандарт зачастую позволяет избежать необходимости указывать тип указателя на массив за счет использования спецификаторов `auto` и `decltype` (см. раздел 2.5.2, стр. 107).

```
// вывести значение каждого элемента ia; каждый внутренний массив
// отображается в отдельной строке
// p указывает на массив из четырех целых чисел
for (auto p = ia; p != ia + 3; ++p) {
    // q указывает на первый элемент массива из четырех целых чисел;
    // т.е. q указывает на int
    for (auto q = *p; q != *p + 4; ++q)
        cout << *q << ' ';
    cout << endl;
}
```

Внешний цикл `for` начинается с инициализации указателя `p` адресом первого массива в массиве `ia`. Этот цикл продолжается, пока не будут обработаны все три ряда массива `ia`. Инкремент `++p` перемещает указатель `p` на следующий ряд (т.е. следующий элемент) массива `ia`.

Внутренний цикл `for` выводит значения внутренних массивов. Он начинается с создания указателя `q` на первый элемент в массиве, на который указывает указатель `p`. Результатом `*p` будет массив из четырех целых чисел. Как

обычно, при использовании имени массива оно автоматически преобразуется в указатель на его первый элемент. Внутренний цикл `for` выполняется до тех пор, пока не будет обработан каждый элемент во внутреннем массиве. Чтобы получить указатель на элемент сразу за концом внутреннего массива, мы снова обращаемся к значению указателя `p`, чтобы получить указатель на первый элемент в этом массиве. Затем добавляем к нему 4, чтобы обработать четыре элемента в каждом внутреннем массиве.

Конечно, используя библиотечные функции `begin()` и `end()` (см. раздел 3.5.3, стр. 168), этот цикл можно существенно упростить:

```
// p указывает на первый массив в ia
for (auto p = begin(ia); p != end(ia); ++p) {
    // q указывает на первый элемент во внутреннем массиве
    for (auto q = begin(*p); q != end(*p); ++q)
        cout << *q << ' '; // выводит значение, указываемое q
    cout << endl;
}
```

Спецификатор `auto` позволяет библиотеке самостоятельно определить конечный указатель и избавить от необходимости писать тип, значение которого возвращает функция `begin()`. Во внешнем цикле этот тип — указатель на массив из четырех целых чисел. Во внутреннем цикле этот тип — указатель на тип `int`.

Псевдонимы типов упрощают указатели на многомерные массивы

Псевдоним типа (см. раздел 2.5.1, стр. 106) может еще больше облегчить чтение, написание и понимание указателей на многомерные массивы. Рассмотрим пример.

```
using int_array = int[4]; // объявление псевдонима типа нового стиля;
                          // см. раздел 2.5.1, стр. 106
typedef int int_array[4]; // эквивалентное объявление typedef;
                          // см. раздел 2.5.1, стр. 106
// вывести значение каждого элемента ia; каждый внутренний массив
// отображается в отдельной строке
for (int_array *p = ia; p != ia + 3; ++p) {
    for (int *q = *p; q != *p + 4; ++q)
        cout << *q << ' ';
    cout << endl;
}
```

Код начинается с определения `int_array` как имени для типа “массив из четырех целых чисел”. Это имя типа используется для определения управляющей переменной внешнего цикла `for`.

Упражнения раздела 3.6

Упражнение 3.43. Напишите три разных версии программы для вывода элементов массива `ia`. Одна версия должна использовать для управления перебором серийный оператор `for`, а другие две — обычный цикл `for`, но в одном случае использовать индексирование, а в другом — указатели. Во всех трех программах пишите все типы явно, т.е. не используйте псевдонимы типов и спецификаторы `auto` или `decltype` для упрощения кода.

Упражнение 3.44. Перепишите программы из предыдущего упражнения, используя псевдоним для типа управляющих переменных цикла.

Упражнение 3.45. Перепишите программы снова, на сей раз используя спецификатор `auto`.

Резюме

Одними из важнейших библиотечных типов являются `vector` и `string`. Строка — это последовательность символов переменной длины, а вектор — контейнер объектов единого типа.

Итераторы обеспечивают косвенный доступ к хранящимся в контейнере объектам. Итераторы используются для доступа и перемещения между элементами в строках и векторах.

Массивы и указатели на элементы массива обеспечивают низкоуровневые аналоги библиотечных типов `vector` и `string`. Как правило, предпочтительней использовать библиотечные классы, а не их низкоуровневые альтернативы, массивы и указатели, встроенные в язык.

Термины

Арифметические действия с итераторами (*iterator arithmetic*). Операции с итераторами векторов и строк. Добавление и вычитание целого числа из итератора приводит к изменению позиции итератора на соответствующее количество элементов вперед или назад от исходного. Вычитание двух итераторов позволяет вычислить дистанцию между ними. Арифметические действия допустимы лишь для итераторов, относящихся к элементам того же контейнера.

Арифметические действия с указателями (*pointer arithmetic*). Арифметические операции, допустимые для указателей. Указатели на массивы поддерживают те же операции, что и арифметические действия с итераторами.

Индекс (*index*). Значение, используемое в операторе индексирования для указания элемента, возвращаемого из строки, вектора или массива.

Инициализация значения (value initialization). Инициализация, в ходе которой объекты встроенного типа инициализируются нулем, а объекты класса — при помощи стандартного конструктора класса. Объекты типа класса могут быть инициализированы значением, только если у класса есть стандартный конструктор. Используется при инициализации элементов контейнера, когда указан его размер, но не указан инициализирующий элемент. Элементы инициализируются копией значения, созданного компилятором.

Инициализация копией (copy initialization). Форма инициализации, использующая знак `=`. Вновь созданный объект является копией предоставленного инициализатора.

Итератор после конца (off-the-end iterator). Итератор, возвращаемый функцией `end()`. Он указывает не на последний существующий элемент контейнера, а на позицию за его концом, т.е. на несуществующий элемент.

Контейнер (container). Тип, объекты которого способны содержать коллекцию объектов определенного типа. К контейнерным относится тип `vector`.

Объявление `using`. Позволяет сделать имя, определенное в пространстве имен, доступным непосредственно в коде. `using пространствоимен::имя;`. Теперь `имя` можно использовать без префикса `пространствоимен::`.

Оператор `!`. Оператор логического NOT. Возвращает инверсное значение своего операнда типа `bool`. Результат `true`, если операнд `false`, и наоборот.

Оператор `&&`. Оператор логического AND. Результат `true`, если оба операнда `true`. Правый операнд обрабатывается, *только если* левый операнд `true`.

Оператор `[]`. Оператор индексирования. Оператор `obj[i]` возвращает элемент в позиции `i` объекта контейнера `obj`. Счет индексов начинается с нуля: первый элемент имеет индекс 0, а последний — `obj.size() - 1`. Индексирование возвращает объект. Если `p` — указатель, а `n` — целое число, то `p[n]` является синонимом для `*(p+n)`.

Оператор `||`. Оператор логического OR. Результат `true`, если любой операнд `true`. Правый операнд обрабатывается, *только если* левый операнд `false`.

Оператор `++`. Для типов итераторов и указателей определен оператор инкремента, который “добавляет единицу”, перемещая итератор или указатель на следующий элемент.

Оператор `<<`. Библиотечный тип `string` определяет оператор вывода, читающий символы в строку.

Оператор `->`. Оператор стрелка. Объединяет оператор обращения к значению и точечный оператор: `a->b` — синоним для `(*a).b`.

Оператор `>>`. Библиотечный тип `string` определяет оператор ввода, читающий разграниченные пробелами последовательности символов и сохраняющий их в строковой переменной, указанной правым операндом.

Серийный оператор `for` (range for). Управляющий оператор, перебирающий значения указанной коллекции и выполняющий некую операцию с каждым из них.

Переполнение буфера (buffer overflow). Грубая ошибка программирования, результат использования индекса, выходящего из диапазона элементов контейнера, такого как `string`, `vector` или массив.

Прямая инициализация (direct initialization). Форма инициализации, не использующая знак `=`.

Расширение компилятора (compiler extension). Дополнительный компонент языка, предлагаемый некоторыми компиляторами. Код, применяющий расширение компилятора, может не подлежать переносу на другие компиляторы.

Создание экземпляра (instantiation). Процесс, в ходе которого компилятор создает специфический экземпляр шаблона класса или функции.

Строка в стиле C (C-style string). Символьный массив с нулевым символом в конце. Строковые литералы являются строками в стиле C. Строки в стиле C могут стать причиной ошибок.

Строка с завершающим нулевым символом (null-terminated string). Строка, последний символ которой сопровождается нулевым символом (`'\0'`).

Тип `difference_type`. Целочисленный знаковый тип, определенный в классах `vector` и `string`, способный содержать дистанцию между любыми двумя итераторами.

Тип `iterator` (итератор). Тип, используемый при переборе элементов контейнера и обращении к ним.

Тип `ptrdiff_t`. Машинозависимый знаковый целочисленный тип, определенный в заголовке `cstdint`. Является достаточно большим, чтобы содержать разницу между двумя указателями в самом большом массиве.

Тип `size_t`. Машинозависимый беззнаковый целочисленный тип, определенный в заголовке `cstdint`. Является достаточно большим, чтобы содержать размер самого большого возможного массива.

Тип `size_type`. Имя типа, определенного для классов `vector` и `string`, способного содержать размер любой строки или вектора соответственно. Библиотечные классы, определяющие тип `size_type`, относят его к типу `unsigned`.

Тип `string`. Библиотечный тип, представлявший последовательность символов.

Тип `vector`. Библиотечный тип, содержащий коллекцию элементов определенного типа.

Функция `begin()`. Функция-член классов `vector` и `string`, возвращающая итератор на первый элемент. Кроме того, автономная библиотечная функция, получающая массив и возвращающая указатель на первый элемент в массиве.

Функция `empty()`. Функция-член классов `vector` и `string`. Возвращает логическое значение (типа `bool`) `true`, если размер нулевой, или значение `false` в противном случае.

Функция `end()`. Функция-член классов `vector` и `string`, возвращающая итератор на элемент после последнего элемента контейнера. Кроме того, автономная библиотечная функция, получающая массив и возвращающая указатель на элемент после последнего в массиве.

Функция `getline()`. Определенная в заголовке `string` функция, которой передают поток `istream` и строковую переменную. Функция читает данные из потока до тех пор, пока не встретится символ новой строки, а прочитанное сохраняет в строковой переменной. Функция возвращает поток `istream`. Символ новой строки в прочитанных данных отбрасывается.

Функция `push_back()`. Функция-член класса `vector`, добавляющая элементы в его конец.

Функция `size()`. Функция-член классов `vector` и `string` возвращает количество символов или элементов соответственно. Возвращаемое значение имеет тип `size_type` для данного типа.

Шаблон класса (`class template`). Проект, согласно которому может быть создано множество специализированных классов. Чтобы применить шаблон класса, необходимо указать дополнительную информацию. Например, чтобы определить вектор, указывают тип его элемента: `vector<int>` содержит целые числа.